Privacy-Preserving Federated Learning Scheme With Mitigating Model Poisoning Attacks: Vulnerabilities and Countermeasures

Jiahui Wu, Fucai Luo, Tiecheng Sun, *Member, IEEE*, Haiyan Wang, and Weizhe Zhang, *Senior Member, IEEE*

***Abstract*—The privacy-preserving federated learning schemes based on the setting of two honest-but-curious and non-colluding servers offer promising solutions in terms of security and efficiency. However, our investigation reveals that these schemes still suffer from privacy leakage when considering model poisoning attacks from malicious users. Specifically, we demonstrate that the privacy-preserving computation process for defending against model poisoning attacks inadvertently leaks privacy to one of the honest-but-curious servers, enabling it to access users' gradients in plaintext. To address this issue, we propose an enhanced privacy-preserving and Byzantine-robust federated learning (PBFL) framework that simultaneously achieves privacy, robustness, and efficiency. Central to our design is a novel Byzantine-tolerant aggregation strategy that defends against both conventional and adaptive poisoning attacks. It integrates normalization judgment, cosine similarity computation, and adaptive user weighting, with a dual-scoring trust mechanism and outlier suppression for stealthy attacks. In addition, we develop two privacy-preserving subroutines, namely secure normalization judgment and secure cosine similarity measurement, which operate over encrypted gradients using a trapdoor fully homomorphic encryption (FHE) scheme, ensuring both confidentiality and robust aggregation correctness. Theoretical analyses confirm that our scheme guarantees security, convergence, and efficiency even with malicious users and one malicious server. Extensive experiments demonstrate that our method effectively breaks prior privacy attacks, maintains high accuracy under diverse poisoning strategies, and significantly reduces computation and communication overhead compared to state-of-the-art PBFL schemes.**

***Index Terms*—Federated learning (FL), privacy protection, poisoning attacks, homomorphic encryption.**

Received 28 October 2024; revised 8 August 2025; accepted 22 September 2025. Date of publication 2 October 2025; date of current version 14 January 2026. This work was supported in part in part by the National Key Research and Development Program of China under Grant 2025YFE0200100 and in part by the Key Program of the Joint Fund of the National Natural Science Foundation of China under Grant U22A2036. *(Corresponding author: Weizhe Zhang.)*

Jiahui Wu, Tiecheng Sun, and Haiyan Wang are with New Network Department, Peng Cheng Laboratory, Shenzhen 518000, China (e-mail: wujh01@pcl.ac.cn; tiechengsun@126.com; wanghy01@pcl.ac.cn).

Fucai Luo is with the School of Computer Science and Technology, Zhejiang Gongshang University, Hangzhou 310018, China (e-mail: lfucai@126.com).

Weizhe Zhang is with the School of Cyberspace Science Faculty of Computing, Harbin Institute of Technology, Shenzhen 518055, China, and also with New Network Department, Peng Cheng Laboratory, Shenzhen 518000, China (e-mail: wzzhang@hit.edu.cn).

This article has supplementary downloadable material available at https://doi.org/10.1109/TDSC.2025.3617070, provided by the authors.

Digital Object Identifier 10.1109/TDSC.2025.3617070

I. Introduction

FEDERATED learning (FL) is a new distributed learning paradigm, which enables collaborative model training among a group of distributed users and a central aggregation server, while keeping users' sensitive data local. In an ideal scenario free from distrustful entities, FL empowers users to retain complete control over their data, thereby facilitating the development of high-quality machine learning models. However, in practice, both servers and users may lack absolute trustworthiness, leading to various attacks [1], [2], [3], [4], [5], [6] against the FL framework. In recent years, the US government has been actively promoting research aimed at creating robust and privacy-preserving FL solutions for diverse tasks such as financial crime prevention [7] and pandemic response and forecasting [8]. Incentives, including substantial prizes totaling $\$800,000$, have been offered to spur innovation in this area. Over the years, extensive researches [9], [10], [11], [12], [13], [14], [15], [16], [17], [18] have underscored the importance of two critical security properties in FL:

- *Privacy protection of gradients:* In FL, the central server may recover sensitive information of the users' training datasets by analyzing the local gradients received from users [1], [2], [3]. To resist privacy leaks from gradients, privacy-preserving federated learning (PPFL) techniques [9], [10], [11], [12], [13] are developed. These techniques involve users masking their local gradients using privacy protection methods before submitting them to the server for secure aggregation.
- *Mitigation of poisoning gradients:* Malicious users may be manipulated by Byzantine adversaries to send poisonous local gradients, deviating from correct model training. The adversarial objective may involve forcing the model to classify inputs into specific incorrect classes (targeted attacks) [4], [5] or inducing misclassifications without specifying a particular target class (untargeted attacks) [6]. Such model poisoning attacks can severely compromise the model's performance. Recent works [15], [16], [17], [18] show that cosine similarity is an effective method to detect poisonous gradients.

The existing solutions for providing privacy and defending against poisoning attacks in FL focus on two opposing directions: privacy-preserving FL solutions aim to achieve data

1545-5971 © 2025 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Anhui University. Downloaded on June 16,2026 at 03:35:52 UTC from IEEE Xplore. Restrictions apply.

TABLE I
QUALITATIVE COMPARISON OF FL REPRESENTATIVE WORKS

Schemes	Confide.	Poison. Defense	Server Setting	Correct.	Effic.
FedSGD [33]	○	○	1	●	●
PrivateFL [9]–[13]	●	○	0/1/2	●	●
RobustFL [14]–[18]	○	●	0/1/2	●	●
PBFL [22]–[24]	○	●	0	●	○
PBFL [19]–[21]	○	●	1	●	○
PBFL [30]–[32]	◐	●	1	●	●
PBFL [28], [29]	◐	●	2	●	●
PBFL [25]–[27]	○	●	2	○	●
Our scheme	●	●	2	●	●

Notes. Confide., Poison., Correct., and Effic. denote *confidentiality*, *poisoning*, *correctness*, and *efficiency*, respectively. ●= satisfied, ○= not satisfied, ◐= partially satisfied. In the column of "Server Setting", 0, 1, and 2 represent the server-less, the single-server, and the two-server models, respectively.

indistinguishability, whereas defenses against distorted gradients often rely on the similarity between malicious and benign gradients to filter out malicious ones. Specifically, defenses that extract valuable statistical information largely depend on the distinguishability of the data. Therefore, simultaneously achieving both security goals in FL remains challenging. Recently, some efforts have been made to simultaneously achieve both objectives. For instance, researchers propose privacy-preserving and Byzantine-robust FL (PBFL) schemes. However, these schemes often suffer from inefficiency [19], [20], [21], [22], [23], [24], may still incur privacy leakages [22], [23], [24], [25], [26], [27], [28], [29], are not secure for secure multiparty computation (MPC) scenarios like PBFL applications [30], or rely on a fully trusted third party that must remain involved throughout the entire training process to decrypt user ciphertexts and perform poisoning detection [31], [32]. Based on server aggregation settings, we classify existing PBFL schemes into three categories: the serverless/decentralized model, the single-server model, and the two-server model. Table I presents a qualitative comparison of FL representative works with different server aggregation settings.

In the serverless/decentralized model, multiple users collaborate in training a model without any central server. Blockchain techniques are commonly employed to resist model poisoning attacks in FL with the serverless setting (e.g., [22], [23], [24]) since it provides a distributed and transparent framework for storing and managing model updates in FL, thereby reducing the risk of model poisoning attacks. However, these schemes bring additional latency due to the consensus process and result in exponentially increasing communication costs among users, thereby diminishing the efficiency of FL. Moreover, these schemes encounter challenges in privacy protection because of the openness and transparency of the data stored on the blockchain, and the introduction of privacy protection technology will bring higher storage costs and may be limited by low transaction throughput.

FL schemes with the single-server model [19], [20], [21], [30], [31], [32] rely on a central server to perform gradient aggregation. Most of these schemes typically employ privacy protection techniques based on multiple keys/parameters to ensure reliable privacy protection among users, such as local sensitivities of local differential privacy in [19] and individual keys of multi-key CKKS [34] in [20], [21]. Although these methods offer certain privacy guarantees, they often suffer from low efficiency. In particular, multi-key CKKS-based schemes incur high computational and communication costs, with overall complexity reaching $O(n^2)$, where n is the number of users [35]. More critically, recent analyses [36], [37] reveal that state-of-the-art multi-key CKKS schemes, including [34], [38], are vulnerable to gradient leakage during distributed decryption, thereby undermining their core security guarantees. To improve scalability, more recent works [30], [31], [32] adopt single-key CKKS [39]. In [30], all users use the same encryption and decryption key pair. This design enables efficient aggregation with a computational complexity of only $O(n)$. However, it fundamentally compromises inter-user confidentiality, as any user with access to the shared decryption key can decrypt ciphertexts submitted by other users. This limitation becomes particularly critical under the PBFL threat model, which explicitly accounts for the presence of malicious users capable of exploiting such vulnerabilities. In [31], [32], each user encrypts its local gradient using either a third-party public key or its own individual key. However, to enable subsequent malicious update detection, a trusted third party is required to decrypt the received ciphertexts and perform poisoning detection on the plaintext gradients. This third party must be fully involved in every round of training, which is impractical in realistic FL settings. More fundamentally, if such a trusted party exists and is assumed to have full decryption capabilities, then the use of HE becomes unnecessary. Instead, users could simply encrypt their gradients using traditional public-key encryption schemes and transmit them directly to the trusted entity. This alternative would not only simplify the protocol but also significantly reduce computational overhead. Therefore, the assumption of a fully trusted and continuously online detection effectively undermines the motivation for HE in these schemes. As such, the overall design of these approaches is fundamentally flawed and unrealistic for deployment in adversarial or semi-honest federated environments. Additionally, relying on a single server introduces a single point of failure, exposing the system to a wide range of external threats, such as hacking, data breaches, or unintended information leakage, all of which could compromise both model privacy and system reliability.

FL schemes with the two-server model (e.g., [25], [26], [27], [28], [29]) distribute the trust assumed in the traditional single-server model across two non-colluding servers. These schemes are promising as they eliminate single points of failure and enhance robustness. To preserve the confidentiality of user gradients, these schemes typically combine HE with random masking to construct secure poisoning detection protocols. In particular, one of the two non-colluding servers adds a random mask to the received ciphertexts and then engages in secure detection protocols with the other server. Compared to single-server HE-based PBFL models, these designs do not require a fully trusted third party and ensure that users can only decrypt the final aggregated model, not the local ciphertexts of others. As a result, they achieve inter-user confidentiality with significantly

Authorized licensed use limited to: Anhui University. Downloaded on June 16,2026 at 03:35:52 UTC from IEEE Xplore. Restrictions apply.

TABLE II
THE NOTATIONS AND THEIR SEMANTIC MEANINGS

Nots.	Meanings	Nots.	Meanings
n	The number of the users	T	The number of the iterations
ϵ, ϵ'	Security parameters	p_1, p_2	Two odd primes
p, q	Plaintext/ciphertext modulus	P	Special modulus
n_f	Ring dimension	$\mathcal{R}_q$	Cyclotomic ring
$\mathbb{R}, \mathbb{C}$	Real/complex number field	χ_s, χ_e	Probability distribution
pk, sk, evk	Public/secret/evaluation key	sk_i	Secret key shares
$[\![\cdot]\!]$, $[\cdot]_i$	Ciphertext/partially decrypted ciphertext	$W^{(t)}$	Model parameter, a flattened vector of all weight matrixes in a learning model
$\|\| \cdot \||$	l_2 norm of a vector	cos	Cosine similarity
co_i, cs_i	Confidence/credit score of U_i	η	Learning rate of gradient descent method
$g_i^{(t)}$, $g_\Sigma^{(t)}$	Local/global gradient vector	$g_*^{(t)}$	Poisonous gradient vector
α	Parameter of Dirichlet distribution	Att_{ro}	Attack ratio: the proportion of malicious users to the total number of users

lower overhead, typically with only $O(n)$ complexity. However, in these schemes, preserving *computational correctness* while maintaining *data confidentiality* remains a fundamental challenge. For example, in ShieldFL [26] and the schemes by Miao et al. [28], [29], two servers collaboratively decrypt and aggregate the encrypted gradient updates from users. To prevent either server from directly accessing the plaintext gradients, a masking mechanism is employed in which one server adds a random noise vector to each gradient ciphertext before it is decrypted by the other server. Malicious update detection is then carried out on the decrypted masked gradients. In practice, the same random noise vector is applied to all gradient ciphertexts to ensure that the noise can be removed consistently and computational correctness can be preserved. This uniform masking, however, unintentionally exposes the relative differences between gradients. As a result, an honest-but-curious server that is also registered as a user can subtract its own known gradient and the shared noise vector to reconstruct the exact gradients of other users, thereby compromising privacy. Furthermore, a malicious server can recover all honest users' gradients by colluding with a malicious user. Using distinct noise vectors for each user could mitigate this vulnerability, but it would break computational correctness, since scalar values required for poisoning detection, such as cosine similarity scores computed from differently masked gradients, cannot be demasked from the aggregated scalar outputs. This example illustrates a fundamental tension in current two-server FL designs: the requirement to preserve **computational correctness** for accurate robust aggregation often conflicts with the goal of ensuring **data confidentiality**.

In light of these challenges, this paper makes the following key contributions:

1) We conduct a security analysis of an existing two-trapdoor HE-based PBFL scheme under the two-server setting and reveal a critical privacy vulnerability. Specifically, we demonstrate that the main secure computation procedure of the scheme leaks substantial gradient information to an untrusted server. This flaw allows the server to potentially reconstruct the plaintext gradients of all users, thereby compromising the confidentiality of the poisoning defense mechanism.
2) We design a novel Byzantine-tolerant aggregation strategy that remains robust against both conventional model poisoning attacks (e.g., label flipping and random perturbation) and stronger adaptive poisoning attacks. The strategy consists of three core components: normalization judgment, cosine similarity computation, and adaptive user weighting. It incorporates a dual-scoring trust mechanism that combines short-term confidence and long-term credit scores, softmax-based similarity normalization, and adaptive user filtering. To defend against stealthy, tailored attacks such as AGR-tailored and AGR-agnostic strategies, we introduce an outlier suppression module based on trust-weight drop detection and differentiated credit score adjustments. Our aggregation method maintains strong resilience against poisoning attacks, even in federated learning scenarios with heterogeneous, non-IID user data.
3) Building on this defense, we propose an enhanced PBFL scheme that provides joint guarantees of multiparty security, communication and computation efficiency, and robust aggregation correctness. To enable privacy-preserving execution of the aggregation logic, we develop secure implementations of the normalization judgment and cosine similarity subroutines, allowing them to operate over encrypted gradients without leaking sensitive information. To accommodate stronger adversarial settings, the scheme is further extended to tolerate arbitrarily malicious users and up to one malicious, non-colluding server. This enhancement is achieved by upgrading the decryption phase from a two-party protocol (between the user and a single server) to a three-party protocol involving the user and both servers. In addition, by assigning each user an independent decryption key, our scheme ensures multiparty security and effectively prevents cross-user information leakage.
4) We implement our scheme and conduct comprehensive experiments to evaluate its performance in terms of privacy, robustness, and efficiency. The results demonstrate that: (i) our proposed privacy attack effectively compromises existing two-server PBFL schemes; (ii) our enhanced aggregation strategy maintains high robustness under various poisoning attacks, including adaptive threats; and (iii) our scheme significantly outperforms state-of-the-art PBFL baselines in both computation and communication overhead.

The remainder of this paper is organized as follows. In Section II, we introduce the related PBFL schemes. In Section III, we introduce the system model, the threat model, and the design goals. In Section IV, we overview the two-server-based PBFL scheme ShieldFL and present our privacy attacks to ShieldFL. To defeat our attacks and achieve design goals, an enhanced scheme is proposed in Section V. Section VI analyzes the security, complexity, and convergence of the enhanced scheme. Section VII introduces the experiments to verify the effectiveness of our attack on ShieldFL and the effectiveness of our enhanced scheme

Authorized licensed use limited to: Anhui University. Downloaded on June 16,2026 at 03:35:52 UTC from IEEE Xplore. Restrictions apply.

against model poisoning attacks and efficiency metrics. Finally, conclusions are given in Section VIII. We give the necessary notations in Table II.

II. Related Work

FL with the server-less/decentralized model: Blockchain techniques are usually employed to defend against poisoning attacks in this model since it provides a distributed and transparent framework for storing and managing model updates in FL, thus reducing the risk of such attacks. Kasyap et al. [22] propose a new Blockchain consensus mechanism based on honest users' contributions on FL training, incentivizing positive training behaviors and thereby reducing the likelihood of model poisoning attacks. Xu et al. [23] propose recording an authenticator of the training program and global model on a Blockchain to resist model poisoning attacks. This scheme allows FL users to verify the global models they receive by querying the authenticator. Meanwhile, Mohamed et al. [24] design an incentive strategy that rewards reputation values to each participating user, records these values on a blockchain, and empowers users to choose users with high historical reputations for model aggregation. These schemes preserve the integrity of the federated model against poisoning attacks, but they introduce additional latency from the consensus process and incur exponentially increasing communication costs between the users, thereby reducing FL efficiency. Moreover, these schemes still face privacy protection challenges due to the openness and transparency of blockchain-stored data. Introducing privacy protection technology may escalate storage costs and could be hindered by low transaction throughput.

FL with single-server model: In general, existing researches on the single-server model achieve the secure computations of Byzantine attack detection and model aggregation by directly combining general privacy-enhancing techniques or infrastructures such as differential privacy (DP) [19], [40], homomorphic encryption (HE) [20], [21], [30], [31], [32], [41], secure multiparty computation (MPC) [42], [43], [44], [45], [46], and hardware-based trusted execution environment (TEE) [47], [48], [49]. In these schemes, the different users masked their local gradients using either a unique parameter/key (e.g., a sensitivity of global DP in [40] and a secret key of single-key HE in [41]) or using their individual parameters/keys (e.g., local sensitivities of local DP in [19] and individual keys of multi-key HE [20], [21]). The latter approach provides more reliable privacy protection but may suffer from high performance degradation, including increased communication/computation burdens and lossy model accuracy, when dealing with multi-parameter/key masked gradients. For example, the state-of-the-art practical multi-key CKKS schemes [34], [38] own computational and communication complexity of $O(n^2)$, where n is the number of users [35]. More critically, recent analyses [36], [37] reveal that these multi-key CKKS schemes [34], [38] are vulnerable to gradient leakage during distributed decryption, thereby undermining their core security guarantees. To improve scalability, more recent works [30], [31], [32] adopt single-key CKKS [39], where all users encrypt their data under a shared key. This allows efficient aggregation with only $O(n)$ complexity. However, such approaches fundamentally fail to ensure inter-user confidentiality, as all ciphertexts can be decrypted by any party with access to the shared key.

MPC-based PBFL schemes [42], [43], [44], [45], [46] naturally ensure secure multi-party computation among multiple users. Most of these schemes adopt secret sharing to achieve secure aggregation, where each user splits its private gradient into multiple random shares and engages in a secure multiparty robust aggregation protocol to securely obtain the global gradient. These schemes ensure that only the final aggregated gradient is revealed, while the local gradients of individual users remain confidential. However, such schemes typically have high computational and communication complexities. For example, a recent secret-sharing-based secure aggregation protocol [45] has computational and communication complexities of $O(n^2(\log n)^2)$ and $O(n^2)$, respectively.

TEE-based PBFL schemes [47], [48], [49] rely on hardware-assisted secure enclaves to protect sensitive computations during federated learning. These schemes typically place local training or aggregation inside TEEs such as Intel SGX or ARM TrustZone, preventing gradient leakage from both external observers and the host system. For example, PPFL [47] adopts layer-wise training within TEEs to fit memory limits, while FLATEE [48] leverages secure aggregation and differential privacy to resist poisoning. PELTA [49] shields shallow layers to block white-box adversarial attacks during inference. However, TEE-based schemes require trust in the hardware provider, and remain vulnerable to side-channel attacks and limited by enclave memory.

FL with the multi-server model: FL with the multi-server model, particularly the two-server model, gains attention in recent research on PBFL. In this model, users distribute their protected gradients across multiple non-colluding servers for secure aggregation. This approach mitigates single points of failure and, compared to single-server and server-less models, reduces the computational and communication burden on users without compromising model performance. While protocols with more than two servers can offer better efficiency, ensuring the validity of the honest-majority assumption remains challenges. Conversely, for dishonest-majority protocols, two-server counterparts bring more performance improvements than more servers (see, e.g., comparison in Table 6 of [50]). Liu et al. [25] propose PEFL scheme based on the two-server model. It employs linear HE to protect the privacy of local model updates and designs the corresponding secure similarity computation protocols between the two servers to detect poisonous gradient ciphertexts. However, these secure computation protocols inadvertently disclose data privacy to the servers [51]. Zhang et al. [27] propose LSFL scheme, which utilizes additive secret sharing to design a secret-sharing-based, secure Byzantine-robust aggregation protocol between the two servers. This protocol enables poison detection and secure aggregation. However, this scheme also suffers from privacy leakages during the implementation of secure aggregation between the two servers [52]. Ma et al. [26] propose ShieldFL, a two-server model employing two-trapdoor HE and cosine similarity for encrypting gradients and detecting model poisoning. In ShieldFL, the secret key of the encryption

Authorized licensed use limited to: Anhui University. Downloaded on June 16,2026 at 03:35:52 UTC from IEEE Xplore. Restrictions apply.

algorithm is divided into two secret shares and distributed to two non-colluding servers separately. These servers then collaborate to detect poisonous encrypted gradients and obtain an aggregation gradient using a masking technique. However, this scheme still violates privacy. The privacy breaches of these two-server-based PBFL schemes arise from the secure computation processes of similarity measurements between the two servers. Specifically, the computation involves securely computing the similarity between a local gradient and a baseline gradient (e.g., the median gradient of all gradients in [27] or the poisonous gradient baseline in ShieldFL), with the baseline gradient being identical across multiple computations. This identical baseline gradient makes it easy to design insecure schemes like ShieldFL and LSFL, which compromise privacy. Miao et al. [28], [29] propose a two-server framework combined with CKKS homomorphic encryption to enable secure two-party poisoning detection. In both schemes, one server, for example S_1, applies the *same random mask* to all encrypted gradients and forwards them to the other server S_2 for decryption and detection. These designs achieve high efficiency, with both computation and communication complexities scaling linearly with the number of users $O(n)$. They avoid the inter-user confidentiality weakness inherent in single-key HE based PBFL schemes and remove the need for a fully trusted third party to remain involved throughout training. Their security analysis focuses on the setting with two semi-honest and non-colluding servers in the presence of malicious users; it does not extend to the stronger threat model where some users and one of the two servers may be malicious. This stronger model is explored in recent works such as ELSA [40] and SAFEFL [44]. Both schemes are based on MPC for PBFL, which, while offering strong security guarantees, still face efficiency challenges, particularly with respect to communication overhead.

In this paper, we focus on PBFL schemes utilizing the two-server setting. These schemes fortify resilience against single points of failure and have the potential to improve FL performance regarding computational cost, communication burden, and final model accuracy. While Ma et al. [26] and Zhang et al. [27] have designed their PBFL schemes based on the two-server setting, their approaches unfortunately fall short in preserving privacy. Consequently, our scheme aims to rectify these privacy breaches and further improve computational and communication efficiency.

III. Problem Formulation

A. System Model

Fig. 1 illustrates the system model of our FL framework, comprising three types of entities: a key center *KC*, n users $U_{i\in[1,n]}$, and two servers S_1 and S_2.

- *Key Center: KC* generates and distributes public key and secret key shares to users and servers.
- *Users:* In the tth training iteration, each user U_i trains the local model $W^{(t)}$ with the individual dataset, encrypts its local gradient $g_i^{(t)}$ with the public key, and uploads the encrypted gradient $[\![g_i^{(t)}]\!]$ to S_1. Then, each user downloads the global gradient $g^{(t)}$ for local update.

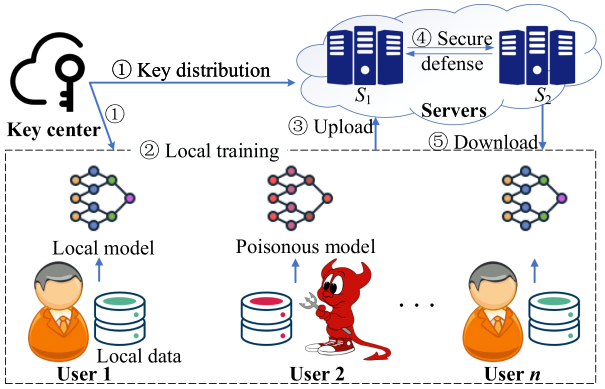


Fig. 1. The system model.

- *Servers:* S_1 and S_2 interact to detect poisoning of the users' local gradients $\{[\![g_i^{(t)}]\!]\}_{i\in[1,n]}$ and perform aggregating of the local gradients.

B. Threat Model

We consider a federated learning system comprising a set of n users $\{U_i\}_{i\in[1,n]}$ and two non-colluding servers S_1, S_2. The servers collaboratively manage encrypted aggregation and filtering tasks, while the uses periodically upload their local updates. We assume the presence of *malicious uses* and at most *one corrupted server*, a widely accepted assumption in dual-server frameworks [40], [44]. To capture different adversarial behaviors, we distinguish between *honest-but-curious* and *malicious* corruption types:

- *Honest-but-curious (semi-honest)* parties follow the protocol specification but attempt to infer private information from the data they receive.
- *Malicious* parties may arbitrarily deviate from the protocol, drop or fabricate messages, or tamper with the computation.

We analyze the following two adversarial scenarios:

Case 1. Semi-honest server with malicious uses: In this setting, both servers behave semi-honest, while some uses may be malicious. The semi-honest servers does not deviate from the protocol but attempts to learn sensitive information. Under this threat model, our scheme ensures:

- The *privacy* of honest uses' local updates (e.g., gradients or model parameters) is preserved.
- The *correctness* of server-side computations, including aggregation and filtering, is guaranteed.
- Malicious uses cannot compromise the privacy of others or inject deviating influence beyond their own inputs.

Case 2. Malicious server with malicious uses: We assume at most one of the two servers is malicious. Without loss of generality, we assume S_2 is corrupted. The corrupted server may alter, drop, or fabricate computation results. Nevertheless, as long as the two servers do not collude, our scheme provides:

- *Privacy protection* of honest uses' updates remains intact due to cryptographic separation and dual control.
- However, the *correctness of computation* cannot be unconditionally guaranteed, as the malicious server may tamper with intermediate results or deviate from the protocol.

Authorized licensed use limited to: Anhui University. Downloaded on June 16,2026 at 03:35:52 UTC from IEEE Xplore. Restrictions apply.

This trade-off is common in dual-server designs [40], [44]. Similar to SAFEFL [44], our framework is compatible with integrating *verifiable computation techniques* (e.g., ZKP, MAC-based audits) to ensure correctness even in the presence of a malicious server. As verifiability introduces additional complexity and is orthogonal to our core design, we leave its implementation as future work.

C. Design Goals

The design goals are to ensure security and performance as detailed below.

- *Confidentiality (Privacy Protection):* The scheme should be resistant to Type I attackers, ensuring that users' private datasets cannot be reconstructed by this type of attackers (including other users and servers).
- *Robustness:* The scheme should be resistant to Type II attackers. It should be capable of detecting model poisoning attacks from malicious users based on users' encrypted local models and should be able to produce a high-quality global model.
- *Efficiency:* The computational and communication overhead on users and servers should be minimized.

IV. Overview of Prior Work

In this section, we briefly review the existing two-server-based PBFL scheme, ShieldFL [26], and identify its security vulnerabilities. ShieldFL employs a two-trapdoor partial homomorphic encryption (PHE) scheme based on Paillier's cryptosystem to preserve data privacy. The specific construction of the two-trapdoor PHE is provided in Appendix A. ShieldFL consists of the following four processes:

1) *Setup:* In this process, KC generates a public/secret key pair (pk, sk) and divides sk into $n+1$ pairs of independent shares (sk_1, sk_2), $\{(sk_{s_i}, sk_{u_i})\}_{i\in[1,n]}$ using $\mathbf{PHE.KeySplit}(sk)$. It then broadcasts pk, distributes sk_1 and sk_2 to S_1 and S_2, respectively, and sends $\{sk_{s_i}\}_{i\in[1,n]}$ and sk_{u_i} to S_1 and each user U_i, respectively.
2) *Local Training:* Each benign user U_i trains its local model $W^{(t)}$, encrypts its local gradient $g_i^{(t)}$, and sends the encrypted gradient $[\![g_i^{(t)}]\!] \leftarrow \mathbf{PHE.Enc}_{pk}(g_i^{(t)})$ to S_1. A malicious user $U_i \in U^*$ may launch model poisoning and send encrypted poisonous gradients $[\![g_{i*}^{(t)}]\!]$ to S_1.
3) *Privacy-Preserving Defense:* This process mainly consists of three implementations. a) Normalization judgment. S_1 determines whether it received encrypted local gradients $[\![g_i^{(t)}]\!]$ are normalized. b) Poisonous Gradient Baseline Finding. S_1 finds the gradient $[\![g_*^{(t)}]\!]$ (among the users' local gradients) with the lowest cosine similarity as the baseline of poisonous gradients through implementing a secure cosine similarity measurement $\cos_{i\Sigma} \leftarrow \mathbf{SecCos}([\![g_i^{(t)}]\!], [\![g_\Sigma^{(t-1)}]\!])$ in Fig. 2, where $[\![g_\Sigma^{(t-1)}]\!]$ is the aggregated gradient of the last iteration. c) Byzantine-Tolerance Aggregation. First, S_1 measures the cosine similarity between the poisonous baseline and each user's local gradient as $\cos_{i*} \leftarrow \mathbf{SecCos}([\![g_i^{(t)}]\!], [\![g_*^{(t)}]\!])$. Then, S_1 sets the confidence $co_i \leftarrow 1 \times deg - \cos_{i*}$ for $g_i^{(t)}$ and calculates $co \leftarrow \sum_{i=1}^n co_i$, $co_i = \lfloor \frac{co_i}{co} \cdot deg \rceil$. Finally, S_1 generates a global gradient ciphertext $[\![g_\Sigma^{(t)}]\!]$, where $g_\Sigma^{(t)} = \sum_{i=1}^n co_i g_i^{(t)}$.
4) *Model Update:* S_1 and U_i interact to perform $\mathbf{PHE.PartDec}$ and $\mathbf{PHE.FullDec}$, so as U_i obtains the global gradient $g_\Sigma^{(t)}$ and updates its local model as $W^{(t+1)} \leftarrow W^{(t)} - \eta^{(t)} g_\Sigma^{(t)}$, where $\eta^{(t)}$ is the learning rate of the tth training iteration.

Given two normalized ciphertexts $[\![g_a]\!] = \{[\![x_{a_1}]\!], \cdots, [\![x_{a_l}]\!]\}$ and $[\![g_b]\!] = \{[\![x_{b_1}]\!], \cdots, [\![x_{b_l}]\!]\}$, S_1 interacts with S_2 to obtain the cosine similarity $\cos_{ab}$ of g_a and g_b:

- @S_1: On receiving $[\![g_a]\!]$ and $[\![g_b]\!]$, S_1 selects l random noises $r_{i\in[1,l]} \leftarrow \mathbb{Z}_N^*$ and masks $[\![g_a]\!]$ and $[\![g_b]\!]$ as $[\![\bar{g}_a]\!] = \{[\![\bar{x}_{a_1}]\!], \cdots, [\![\bar{x}_{a_l}]\!]\}$ and $[\![g_b]\!] = \{[\![\bar{x}_{b_1}]\!], \cdots, [\![\bar{x}_{b_l}]\!]\}$, where $[\![\bar{x}_{a_i}]\!] = [\![x_{a_i}]\!] \cdot [\![r_i]\!]$ and $[\![\bar{x}_{b_i}]\!] = [\![x_{b_i}]\!] \cdot [\![r_i]\!]$. Then, S_1 partially decrypts the masked ciphertexts as $[\bar{x}_{a_i}]_1 \leftarrow \mathbf{PHE.PartDec}_{sk_1}([\![\bar{x}_{a_i}]\!])$ and $[\bar{x}_{b_i}]_1 \leftarrow \mathbf{PHE.PartDec}_{sk_1}([\![\bar{x}_{b_i}]\!])$, $s.t.$, $[\![\bar{x}_{a_i}]\!] \in [\![\bar{g}_a]\!]$, $[\![\bar{x}_{b_i}]\!] \in [\![\bar{g}_b]\!]$, and sends $[\bar{g}_a]_1$, $[\bar{g}_b]_1$, $[\![\bar{g}_a]\!]$, $[\![\bar{g}_b]\!]$ to S_2.
- @S_2: Once obtaining the above ciphertexts, S_2 calls $\mathbf{PHE.PartDec}$ and $\mathbf{PHE.FullDec}$ to perform full decryption and obtain the masked gradients as $\bar{g}_a$ and $\bar{g}_b$. Then S_2 calculates the cosine similarity between $\bar{g}_a$ and $\bar{g}_b$ as $\overline{\cos}_{ab} = \bar{g}_a \odot \bar{g}_b = \sum_{i=1}^m \bar{x}_{a_i} \cdot \bar{x}_{b_i}$, where $\odot$ represents inner product. S_2 calls the encryption algorithm $\mathbf{PHE.Enc}$ to return $[\![\overline{\cos}_{ab}]\!]$ to S_1.
- @S_1: S_1 removes the noise r_i in $[\![\overline{\cos}_{ab}]\!]$ to obtain the final cosine similarity $[\![\cos_{ab}]\!]$ and sends it to S_2.
- @S_2: S_2 computes the decryption share $[\cos_{ab}]_2 \leftarrow \mathbf{PHE.PartDec}_{sk_2}([\![\cos_{ab}]\!])$ and returns it to S_1.
- @S_1: On receiving $[\cos_{ab}]_2$, S_1 obtains the cosine similarity plaintext $\cos_{ab}$ by calling $\mathbf{PHE.PartDec}$ and $\mathbf{PHE.FullDec}$.

Fig. 2. Implementation procedure of $\mathbf{SecCos}([\![g_a]\!], [\![g_b]\!])$.

A. Privacy Leakage in ShieldFL

In ShieldFL, privacy leakages primarily occur during the privacy-preserving defense process. In the procedure of poisonous gradient baseline finding, S_1 interacts with S_2 to measure the secure cosine similarity $\cos_{i\Sigma} \leftarrow \mathbf{SecCos}([\![g_i^{(t)}]\!], [\![g_\Sigma^{(t-1)}]\!])$ for $i \in [1, n]$, so as to find the baseline of poisonous gradient $[\![g_*^{(t)}]\!] \in \{[\![g_i^{(t)}]\!]\}_{i\in[1,n]}$ with the lowest cosine similarity. Specifically, in this procedure, S_1 selects l random noises $\{r_{i1}, \ldots, r_{il}\}$ and masks $[\![g_i^{(t)}]\!] = \{[\![x_{i1}]\!], \ldots, [\![x_{il}]\!]\}$ and $[\![g_\Sigma^{(t-1)}]\!] = \{[\![y_1]\!], \ldots, [\![y_l]\!]\}$ as $[\![\bar{g}_i^{(t)}]\!]$ and $[\![\bar{g}_{\Sigma,i}^{(t)}]\!]$ using (1) and (2), respectively. Then S_1 partially decrypts $[\![\bar{g}_i^{(t)}]\!]$ and $[\![\bar{g}_{\Sigma,i}^{(t)}]\!]$ as $[\bar{g}_i^{(t)}]_1$ and $[\bar{g}_{\Sigma,i}^{(t)}]_1$ and sends $[\![\bar{g}_i^{(t)}]\!]$, $[\![\bar{g}_{\Sigma,i}^{(t)}]\!]$, $[\bar{g}_i^{(t)}]_1$, $[\bar{g}_{\Sigma,i}^{(t)}]_1$ to S_2.

$$\begin{pmatrix} [\![\bar{g}_1^{(t)}]\!] \\ \vdots \\ [\![\bar{g}_i^{(t)}]\!] \\ \vdots \\ [\![\bar{g}_n^{(t)}]\!] \end{pmatrix} = \begin{pmatrix} [\![x_{11}]\!] \cdot [\![r_{11}]\!] & \cdots & [\![x_{1l}]\!] \cdot [\![r_{1l}]\!] \\ \vdots & \cdot & \vdots \\ [\![x_{i1}]\!] \cdot [\![r_{i1}]\!] & \cdots & [\![x_{il}]\!] \cdot [\![r_{il}]\!] \\ \vdots & \cdot & \vdots \\ [\![x_{n1}]\!] \cdot [\![r_{n1}]\!] & \cdots & [\![x_{nl}]\!] \cdot [\![r_{nl}]\!] \end{pmatrix} \quad (1)$$

Authorized licensed use limited to: Anhui University. Downloaded on June 16,2026 at 03:35:52 UTC from IEEE Xplore. Restrictions apply.

$$\begin{pmatrix} [\![\bar{g}_{\Sigma,1}^{(t-1)}]\!] \\ \vdots \\ [\![\bar{g}_{\Sigma,i}^{(t-1)}]\!] \\ \vdots \\ [\![\bar{g}_{\Sigma,n}^{(t-1)}]\!] \end{pmatrix} = \begin{pmatrix} [\![y_1]\!] \cdot [\![r_{11}]\!] & \cdots & [\![y_l]\!] \cdot [\![r_{1l}]\!] \\ \vdots & \cdot & \vdots \\ [\![y_1]\!] \cdot [\![r_{i1}]\!] & \cdots & [\![y_l]\!] \cdot [\![r_{il}]\!] \\ \vdots & \cdot & \vdots \\ [\![y_1]\!] \cdot [\![r_{n1}]\!] & \cdots & [\![y_l]\!] \cdot [\![r_{nl}]\!] \end{pmatrix} \tag{2}$$

Privacy Leakage: S_2 decrypts the ciphertexts received from S_1 to obtain $\bar{g}_i^{(t)}$ and $\bar{g}_{\Sigma,i}^{(t-1)}$ $(i \in [1,n])$ as follows:

$$\begin{pmatrix} \bar{g}_1^{(t)} \\ \vdots \\ \bar{g}_i^{(t)} \\ \vdots \\ \bar{g}_n^{(t)} \end{pmatrix} = \begin{pmatrix} x_{11}+r_{11} & \cdots & x_{1l}+r_{1l} \\ \vdots & \cdot & \vdots \\ x_{i1}+r_{i1} & \cdots & x_{il}+r_{il} \\ \vdots & \cdot & \vdots \\ x_{n1}+r_{n1} & \cdots & x_{nl}+r_{nl} \end{pmatrix} \tag{3}$$

$$\begin{pmatrix} \bar{g}_{\Sigma,1}^{(t-1)} \\ \vdots \\ \bar{g}_{\Sigma,i}^{(t-1)} \\ \vdots \\ \bar{g}_{\Sigma,n}^{(t-1)} \end{pmatrix} = \begin{pmatrix} y_1+r_{11} & \cdots & y_l+r_{1l} \\ \vdots & \cdot & \vdots \\ y_1+r_{i1} & \cdots & y_l+r_{il} \\ \vdots & \cdot & \vdots \\ y_1+r_{n1} & \cdots & y_l+r_{nl} \end{pmatrix} \tag{4}$$

Then, S_2 can learn the difference between two arbitrary local gradients and the difference between the last-iteration aggregated gradient and an arbitrary local gradient:

$$\begin{aligned} d_{ij} &= g_i^{(t)} - g_j^{(t)} = \{x_{i1} - x_{j1}, \ldots, x_{il} - x_{jl}\} \\ &= \left(\bar{g}_i^{(t)} - \bar{g}_j^{(t)}\right) - \left(\bar{g}_{\Sigma,i}^{(t-1)} - \bar{g}_{\Sigma,j}^{(t-1)}\right), i \neq j. \end{aligned} \tag{5}$$

$$\begin{aligned} d_{i\Sigma} &= g_i^{(t)} - g_\Sigma^{(t-1)} = \{x_{i1} - y_1, \ldots, x_{il} - y_l\} \\ &= \bar{g}_i^{(t)} - \bar{g}_{\Sigma,i}^{(t-1)}. \end{aligned} \tag{6}$$

Thus, according to (5) and (6), S_2 knows

$$\begin{pmatrix} g_1^{(t)} \\ \vdots \\ g_i^{(t)} \\ \vdots \\ g_n^{(t)} \end{pmatrix} = \begin{pmatrix} d_{1j} + g_j^{(t)} \\ \vdots \\ d_{ij} + g_j^{(t)} \\ \vdots \\ d_{nj} + g_j^{(t)} \end{pmatrix} = \begin{pmatrix} d_{1\Sigma} + g_\Sigma^{(t-1)} \\ \vdots \\ d_{i\Sigma} + g_\Sigma^{(t-1)} \\ \vdots \\ d_{n\Sigma} + g_\Sigma^{(t-1)} \end{pmatrix}, \tag{7}$$

in which S_2 acquires a "shifted" distribution of all local gradients in plaintext (It is worth noting that $\{d_{ij}\}_{i\in[1,n],j\in[1,n]}$ and $\{d_{i\Sigma}\}_{i\in[1,n]}$ are all accessible to S_2). This compromises privacy significantly [2], as it divulges a considerable amount of information to S_2, thereby failing to fulfill the confidentiality requirement outlined in [26].

The privacy leakage issue in the Byzantine-tolerant aggregation phase is similar to that in the poisonous gradient baseline finding process. For brevity, we present its details in Appendix B. In addition, Appendix B further provides a gradient reconstruction analysis, showing how the leaked information from both phases can be used to recover the complete plaintext gradients.

B. Immediate Fix and Its Ineffectiveness

The main reason for the privacy breaches in ShieldFL is that the local gradients $g_i^{(t)}$ and the last-iteration aggregated gradient $g_\Sigma^{(t-1)}$ (or the poisonous gradient baseline $g_*^{(t)}$) are masked with the same random noise vector, thereby exposing the correlations between these gradients and resulting in the severe consequence that once one of these gradients is leaked, all other gradients can be directly revealed in clear.

Although different random noise vectors can be used to prevent the aforementioned privacy violations, using the same noise vector is inevitable to maintain the correctness of **SecCos**. In detail, the secure cosine similarity measurement between two gradient ciphertexts $[\![g_a]\!]$ and $[\![g_b]\!]$ is calculated by first masking the two gradients with random noise vectors r and γ to output $[\![\bar{g}_a]\!] = [\![g_a + r]\!]$ and $[\![\bar{g}_b]\!] = [\![g_b + \gamma]\!]$, respectively, then decrypting the masked gradients and calculating the cosine similarity $\overline{\cos}_{ab} = \frac{\bar{g}_a \odot \bar{g}_b}{\|\bar{g}_a\| \cdot \|\bar{g}_b\|} = \frac{(g_a+r)\odot(g_b+\gamma)}{\|g_a+r\| \cdot \|g_b+\gamma\|}$. However, $\frac{\bar{g}_a \odot \bar{g}_b}{\|\bar{g}_a\| \cdot \|\bar{g}_b\|} = \frac{g_a \odot g_b}{\|g_a\| \cdot \|g_b\|}$ holds only when $r = \gamma$, where $\frac{g_a \odot g_b}{\|g_a\| \cdot \|g_b\|}$ is the correct computation of the cosine similarity between g_a and g_b; Otherwise, if $r \neq \gamma$, the correctness will be breached.

To prevent privacy violations and ensure the correctness of computations, we deviate from ShieldFL of employing distinct random noise vectors to obscure gradient ciphertexts. Instead, we introduce a novel method where we mask the computation result of gradient ciphertexts with a random noise vector, as detailed in our new secure computation methods outlined in Section V-B. This approach leverages a newly constructed trapdoor Fully Homomorphic Encryption (FHE) algorithm as the fundamental cryptographic building block.

V. The Enhanced Scheme

In this section, we present an enhanced PBFL scheme based on the two-server setting that addresses the privacy leakage issues identified in ShieldFL and guarantees the correctness of privacy-preserving computations. To improve resilience against malicious users, we design a novel Byzantine-tolerant aggregation strategy that defends effectively against both conventional and adaptive model poisoning attacks. The scheme is further adapted to a stronger adversarial model, providing privacy preserving even when one of the servers is malicious. Moreover, it achieves improved computational and communication efficiency, making it suitable for practical deployment.

A. Technical Intuition

We empower our enhanced scheme to resist privacy attacks while ensuring computation correctness: To simultaneously ensure both security and correctness, We fundamentally redesign the secure computation paradigm in our enhanced scheme. Specifically, we introduce a novel masking protocol build upon a newly constructed trapdoor FHE scheme. Instead of masking the inputs, we mask the computation results using different random vectors. This eliminates the need for shared noise and preserves correctness. To handle ciphertext operations under masking, we classify them into linear (e.g., addition, subtraction, scalar multiplication) and non-linear (e.g., multiplication,

Authorized licensed use limited to: Anhui University. Downloaded on June 16,2026 at 03:35:52 UTC from IEEE Xplore. Restrictions apply.

exponentiation) categories. Linear operations on masked ciphertexts yield correctly demaskable results. In contrast, non-linear operations introduce complex noise that is difficult to eliminate. To mitigate this without adding computational overhead, we perform non-linear operations directly on the original ciphertexts at S_1. The results are then masked and sent to S_2, which performs the remaining linear operations and partial decryption. Finally, S_1 removes the mask to obtain the correct plaintext output. In this design, S_1 only observes the final computation results, while S_2 only accesses masked intermediate values. This separation effectively defends against the proposed privacy attacks.

Building on the technical intuition outlined above, we design two secure computation methods between the servers S_1 and S_2: an enhanced secure normalization judgment and an enhanced secure cosine similarity measurement, as introduced in Section V-B. Notably, our design principle can also be extended to develop other secure computation methods, such as secure Euclidean distance, secure mean, and secure median, by separating the computation into linear and non-linear operations and employing our encryption and masking techniques within the two-server interaction framework.

Furthermore, to align privacy preserving design with our malicious threat model, we modify the decryption protocol from a two-party interaction (between the user and a single server, S_1) to a three-party protocol involving the user and both servers (S_1 and S_2). This change ensures that the plaintext of any honest user remains confidential, even in the presence of arbitrarily malicious users and at most one malicious server. In other words, as long as the two servers do not collude, no single malicious party, neither a user nor one server, can compromise the privacy of other honest users' gradients. Our secure computation framework is therefore fully compatible with this stronger adversarial assumption.

We improve both computational and communication efficiency in the enhanced scheme: This leap in efficiency stems from a couple of key insights:

1) *Ciphertext computation simplicity:* In our enhanced scheme, the secure computation process of non-linear operations on ciphertexts – crucial for operations like computing $\cos_{ab} = g_a \odot g_b$ during $\mathbf{SecCos}(\llbracket g_a \rrbracket, \llbracket g_b \rrbracket)$ – is streamlined. Unlike previous methods that relied on Paillier's addition homomorphism and complex masking/unmasking techniques, requiring multiple interactions between the two servers, we construct a trapdoor FHE that allows for direct non-linear operations on ciphertexts. This direct approach significantly reduces both the number of computation steps and the communication rounds, making our secure computation methods more efficient than those in ShieldFL. For example, our secure computation methods (outlined in Figs. 3 and 4) only require two rounds of interaction between S_1 and S_2, whereas the original scheme requires four rounds, reducing the communication burden by (at least) half.
2) *Cryptographic primitive efficiency:* We employ an ring learning with errors (RLWE)-based FHE, specifically the CKKS cryptosystem, to construct our trapdoor FHE scheme. This choice is more efficient than the Paillier cryptosystem-based two-trapdoor PHE. The RLWE-based system, with its smaller ciphertext modulus requirements, offers lower computational overhead and communication traffic per step. For instance, at a 128-bit security level, the two-trapdoor PHE demands a ciphertext modulus N^2 of at least 3072×2 bits, whereas the RLWE-based FHE can operate with a much smaller q of 56 bits (with $n_f = 2048$),

Given a ciphertext $\llbracket g_i \rrbracket = \{\llbracket g_{i,1} \rrbracket, \cdots, \llbracket g_{i,\tau} \rrbracket\}$ of a local gradient g_i, where $\tau = \lceil l/n_f \rceil$ and $g_{i,j} = \{x_{i,j,1}, \cdots, x_{i,j,n_f}\}$ for $j \in [1, \tau]$, S_1 interacts with S_2 to judge whether the local gradient g_i is normalized:

- @S_1: S_1 first calculates the component-wise second power of $\llbracket g_i \rrbracket$ as $\llbracket g_i^2 \rrbracket = \{\llbracket g_{i,1}^2 \rrbracket, \cdots, \llbracket g_{i,\tau}^2 \rrbracket\}$, where $\llbracket g_{i,j}^2 \rrbracket \leftarrow \mathbf{FHE.Mult}_{evk}(\llbracket g_{i,j} \rrbracket, \llbracket g_{i,j} \rrbracket)$ for $j \in [1, \tau]$. To protect gradient privacy, S_1 selects τ random noise polynomials $r_{i,j} = \{r_{i,j,1}, \cdots, r_{i,j,n_f}\} \leftarrow \mathcal{R}_q$ for $j \in [1, \tau]$ and masks $\llbracket g_{i,j}^2 \rrbracket$ with $r_{i,j}$ as $\llbracket \bar{g}_{i,j}^2 \rrbracket = \llbracket g_{i,j}^2 + r_{i,j} \rrbracket \leftarrow \mathbf{FHE.Add}(\llbracket g_{i,j}^2 \rrbracket, \llbracket r_{i,j} \rrbracket)$. Then, S_1 executes $[\bar{g}_{i,j}^2]_1 \leftarrow \mathbf{FHE.PartDec}_{sk_1}(\llbracket \bar{g}_{i,j}^2 \rrbracket)$ for $j \in [1, \tau]$ and sends $\{[\bar{g}_{i,j}^2]_1, \llbracket \bar{g}_{i,j}^2 \rrbracket\}_{j \in [1,\tau]}$ to S_2.
- @S_2: After obtaining the above ciphertexts, S_2 obtains the masked polynomials $\bar{g}_i^2 = \{\bar{g}_{i,1}^2, \cdots, \bar{g}_{i,\tau}^2\}$ by executing $[\bar{g}_{i,j}^2]_2 \leftarrow \mathbf{FHE.PartDec}_{sk_2}(\llbracket \bar{g}_{i,j}^2 \rrbracket)$ and $\bar{g}_{i,j}^2 \leftarrow \mathbf{FHE.FullDec}(\llbracket \bar{g}_{i,j}^2 \rrbracket, [\bar{g}_{i,j}^2]_1, [\bar{g}_{i,j}^2]_2)$ for $j \in [1, \tau]$. According to the multiplicative homomorphic properties of the trapdoor FHE, it is held that $\bar{g}_{i,j}^2 = \{\bar{x}_{i,j,1}^2, \cdots, \bar{x}_{i,j,n_f}^2\}$, where $\bar{x}_{i,j,k}^2 = x_{i,j,k}^2 + r_{j,k}$ for $k \in [1, n_f]$. Then, S_2 returns $\overline{sum} = \sum_{j=1}^{\tau} \sum_{k=1}^{n_f} \bar{x}_{i,j,k}^2$ to S_1.
- @S_1: On receiving $\overline{sum}$, S_1 removes the random noises in $\overline{sum}$ by executing $sum = \overline{sum} - \sum_{j=1}^{\tau} \sum_{k=1}^{n_f} r_{i,j,k}$. Then S_1 determines whether $sum = 1$. If yes, it means that g_i is normalized; otherwise not.

Fig. 3. Implementation procedure of the enhanced secure normalization judgement method $\mathbf{ESecJudge}(\llbracket g_i \rrbracket)$.

Given two normalized ciphertexts $\llbracket g_a \rrbracket = \{\llbracket g_{a,1} \rrbracket, \cdots, \llbracket g_{a,\tau} \rrbracket\}$ and $\llbracket g_b \rrbracket = \{\llbracket g_{b,1} \rrbracket, \cdots, \llbracket g_{b,\tau} \rrbracket\}$ of the gradients g_a and g_b, respectively, where $g_{a,j} = \{x_{a,j,1}, \cdots, x_{a,j,n_f}\}$ and $g_{b,j} = \{x_{b,j,1}, \cdots, x_{b,j,n_f}\}$ for $j \in [1, \tau]$, S_1 interacts with S_2 to obtain the cosine similarity $\cos_{ab}$ of g_a and g_b:

- @S_1: On receiving $\llbracket g_a \rrbracket$ and $\llbracket g_b \rrbracket$, S_1 calculates the product of the two ciphertexts as $\llbracket g_{c,j} \rrbracket = \llbracket g_{a,j} \rrbracket \times \llbracket g_{b,j} \rrbracket \leftarrow \mathbf{FHE.Mult}_{evk}(\llbracket g_{a,j} \rrbracket, \llbracket g_{b,j} \rrbracket)$ for $j \in [1, \tau]$. To protect gradient privacy, S_1 selects τ random noise polynomials $r_j = \{r_{j,1}, \cdots, r_{j,n_f}\} \leftarrow \mathcal{R}_q$ for $j \in [1, \tau]$ and masks $\llbracket g_{c,j} \rrbracket$ with r_j as $\llbracket \bar{g}_{c,j} \rrbracket = \llbracket g_{c,j} + r_j \rrbracket \leftarrow \mathbf{FHE.Add}(\llbracket g_{c,j} \rrbracket, \llbracket r_j \rrbracket)$. Then, S_1 calculates $\llbracket \bar{g}_{c,\Sigma} \rrbracket = \llbracket \sum_{j=1}^{\tau} \bar{g}_{c,j} \rrbracket \leftarrow \mathbf{FHE.Add}(\llbracket \bar{g}_{c,1} \rrbracket, \cdots, \llbracket \bar{g}_{c,\tau} \rrbracket)$ and partially decrypts it as $[\bar{g}_{c,\Sigma}]_1 \leftarrow \mathbf{FHE.PartDec}_{sk_1}(\llbracket \bar{g}_{c,\Sigma} \rrbracket)$. S_1 sends $[\bar{g}_{c,\Sigma}]_1, \llbracket \bar{g}_{c,\Sigma} \rrbracket$ to S_2.
- @S_2: After receiving $[\bar{g}_{c,\Sigma}]_1, \llbracket \bar{g}_{c,\Sigma} \rrbracket$, S_2 obtains $\bar{g}_{c,\Sigma} = \{x_{c,j,1}, \cdots, x_{c,j,n_f}\}$ by calling $\mathbf{FHE.PartDec}$ and $\mathbf{FHE.FullDec}$. According to the additive and multiplicative homomorphic properties of the trapdoor FHE, it is held that $x_{c,j,k} = \sum_{j=1}^{\tau} (x_{a,j,k} x_{b,j,k} + r_{j,k})$. Then, S_2 calculates the masked cosine similarity $\overline{\cos}_{ab} = \sum_{k=1}^{n_f} \sum_{j=1}^{\tau} x_{c,j,k} = \sum_{k=1}^{n_f} \sum_{j=1}^{\tau} (x_{a,j,k} x_{b,j,k} + r_{j,k})$ and sends it to S_1.
- @S_1: On receiving $\overline{\cos}_{ab}$, S_1 removes the random noises in $\overline{\cos}_{ab}$ by executing $\cos_{ab} = \overline{\cos}_{ab} - \sum_{k=1}^{n_f} \sum_{j=1}^{\tau} r_{j,k}$.

Fig. 4. Implementation procedure of the enhanced secure cosine similarity measurement method $\mathbf{ESecCos}(\llbracket g_a \rrbracket, \llbracket g_b \rrbracket)$.

Authorized licensed use limited to: Anhui University. Downloaded on June 16,2026 at 03:35:52 UTC from IEEE Xplore. Restrictions apply.

significantly reducing the computational cost. We provide a detailed performance comparison in Section VI-C.

3) *No additional trade-offs:* Importantly, these improvements do not introduce extra limitations. Our enhanced scheme retains the same flexibility and scalability as ShieldFL because both adopt a two-trapdoor architecture and the same key distribution mechanism. Although CKKS introduces negligible approximation error due to its encoding and encryption, such error is naturally tolerated by stochastic machine learning models and does not impact convergence or accuracy, as verified by our experimental results in Section VII.

In summary, our enhanced scheme introduces significant improvements in both efficiency and security without sacrificing flexibility, accuracy, or scalability. By employing randomly selected noise for masking and categorizing operations into linear and non-linear types, we effectively address the privacy concerns associated with previous methods. The streamlined process for non-linear operations, facilitated by a n-trapdoor FHE based on the RLWE homomorphic encryption scheme, minimizes the complexity and communication overhead typically required. This innovative approach not only ensures the correctness of the computations but also bolsters resistance to privacy attacks, setting a new benchmark for secure computation methods. Ultimately, our advancements position our scheme as a more efficient and secure alternative to existing solutions like ShieldFL.

B. New Secure Computation Methods

Our new secure computation methods consist of an enhanced secure normalization judgment and an enhanced secure cosine similarity measurement. We construct a n-trapdoor FHE scheme to protect the privacy of these two methods. The n-trapdoor FHE is based on RLWE homomorphic encryption (CKKS cryptosystem [39]), mainly containing the following algorithms.

- $\mathbf{FHE.Setup}(1^{\varepsilon'}) \rightarrow PP$: Given the security parameter ε', the public parameter $PP = \{n_f, p, q, P, \chi_s, \chi_e\}$ is output, where n_f is a ring dimension, p is a plaintext modulus, $q = q(\varepsilon')$ is a ciphertext modulus, P is a special modulus, and χ_s, χ_e are two probability distributions on the ring $\mathcal{R}$ with small bounds.
- $\mathbf{FHE.KeyGen}(PP) \rightarrow (sk, pk, evk)$: Given the public parameter PP, a secret key $sk \leftarrow \chi_s$, a public key $pk \leftarrow (-a \cdot sk + e \bmod q, a) \in \mathcal{R}_q^2$, and an evaluation key $evk \leftarrow (-a' \cdot sk + e' + P \cdot sk^2 \bmod P \cdot q, a') \in \mathcal{R}_{P \cdot q}^2$ are output, where $a, a' \leftarrow \mathcal{R}_q := \mathbb{Z}_q/(X^{n_f}+1)$ and $e, e' \leftarrow \chi_e$.
- $\mathbf{FHE.KeySplit}(sk) \rightarrow (sk_1, \ldots, sk_n)$: Given the secret key sk, n secret key shares $sk_1, \ldots, sk_n$ are generated by randomly dividing sk meeting $sk = sk_1 + \cdots + sk_n \bmod q$.
- $\mathbf{FHE.Enc}_{pk}(m) \rightarrow [\![m]\!]$: This algorithm first encodes a float vector $m \in \mathbb{R}^{n_f}$ with length n_f into an integral coefficient polynomials $m(X) \in \mathcal{R}_p$ and then it encrypts $m(X)$ and outputs the ciphertext $[\![m]\!]$. Specifically, the input vector m is first converted to a complex vector $z = (z_1, \ldots, z_{n_f/2}) \in \mathbb{C}^{n_f/2}$, in which every two elements in m form the real and imaginary parts of a complex number. Then, the complex vector z is encoded as $m(X) = \lfloor \Delta \cdot \phi^{-1}(z) \rceil$, where $\Delta > 1$ is a scaling factor and $\phi : R[X]/(X^{n_f}+1) \rightarrow \mathbb{C}^{n_f/2}$ is a ring isomorphism. Finally, the ciphertext $[\![m]\!] = (m + u \cdot pk_0 + e_0 \bmod q, u \cdot pk_1 + e_1 \bmod q) \triangleq (c_0, c_1) \in \mathcal{R}_q^2$ is output, where $u \leftarrow \chi_s$ and $e_0, e_1 \leftarrow \chi_e$.
- $\mathbf{FHE.Add}([\![m_1]\!], [\![m_2]\!]) \rightarrow [\![m]\!]$: Given two ciphertexts $[\![m_1]\!], [\![m_2]\!]$, the sum $[\![m]\!] = [\![m_1]\!] + [\![m_2]\!] = [\![m_1 + m_2]\!]$ of the two ciphertexts is output.
- $\mathbf{FHE.Mult}_{evk}([\![m_1]\!], [\![m_2]\!]) \rightarrow [\![m]\!]$: Given two ciphertexts $[\![m_1]\!] = (c_0, c_1), [\![m_2]\!] = (c_0', c_1')$, let $(d_0, d_1, d_2) = (c_0c_0', c_0c_1' + c_0'c_1, c_1c_1') \bmod q$, the product $[\![m]\!] = (d_0, d_1) + \lfloor P^{-1} \cdot d_2 \cdot evk \rceil$ of the two ciphertexts is output.
- $\mathbf{FHE.PartDec}_{sk_i}([\![m]\!]) \rightarrow [m]_i$: Given a ciphertext $[\![m]\!] = (c_0, c_1)$, a partial decryption result $[m]_i = c_1 sk_i + e_i \bmod q$ is output, where $e_i \leftarrow \chi_e$.
- $\mathbf{FHE.FullDec}([\![m]\!], [m]_1, \ldots, [m]_n) \rightarrow m$: Given a ciphertext $[\![m]\!] = (c_0, c_1)$ and its partial decryption result pair $([m]_1, \ldots, [m]_n)$, the algorithm first fully decrypts the ciphertext $[\![m]\!]$ as a plaintext polynomial $m(X) = c_0 + [m]_1 + \cdots + [m]_n \bmod p \in \mathcal{R}_p$, then it decodes $m(X)$ by first computing the complex vector $z = \Delta^{-1} \cdot \phi(m(X)) \in \mathbb{C}^{n_f/2}$ and then converting z to a float vector $m \in \mathbb{R}^{n_f}$. Finally, the algorithm outputs m.

Based on the trapdoor FHE scheme, we design the enhanced secure normalization judgment method **ESecJudge** and the enhanced secure cosine similarity measurement method **ESecCos** as Figs. 3 and 4, respectively.

For **ESecJudge**, on inputting a gradient ciphertext $[\![g_i]\!] = \{[\![g_{i,1}]\!], \ldots, [\![g_{i,\tau}]\!]\}$, S_1 interacts with S_2 to judge whether the local gradient g_i is normalized, i.e., whether $[\![g_{i,1}^2]\!] + \cdots + [\![g_{i,\tau}^2]\!] = [\![1]\!]$. In this ciphertext computation, the linear and non-linear operations correspond to additions and multiplications, respectively. We leverage S_1 to perform multiplication operations locally. The resulting computations are then masked and sent to S_2 for the addition operations.

Specifically, S_1 first computes ciphertext multiplications $[\![g_{i,j}^2]\!] \leftarrow \mathbf{FHE.Mult}_{evk}([\![g_{i,j}]\!], [\![g_{i,j}]\!])$ for $j \in [1, \tau]$ to obtain $[\![g_i^2]\!] = \{[\![g_{i,1}^2]\!], \ldots, [\![g_{i,\tau}^2]\!]\}$. It then masks the result $[\![g_i^2]\!]$ as $[\![\bar{g}_i^2]\!] = \{[\![\bar{g}_{i,1}^2]\!], \ldots, [\![\bar{g}_{i,\tau}^2]\!]\}$, where $[\![\bar{g}_{i,j}^2]\!] = [\![g_{i,j}^2 + r_{i,j}]\!] \leftarrow \mathbf{FHE.Add}([\![g_{i,j}^2]\!], [\![r_{i,j}]\!])$ and $r_{i,j}$ is a randomly selected noise polynomial. S_1 partially decrypts $\{[\![\bar{g}_{i,j}^2]\!]\}_{j \in [1,\tau]}$ and sends $\{[\bar{g}_{i,j}^2]_1, [\![\bar{g}_{i,j}^2]\!]\}_{j \in [1,\tau]}$ to S_2. After performing full decryption, S_2 obtains the masked gradient $\bar{g}_i^2 = \{\bar{g}_{i,1}^2, \ldots, \bar{g}_{i,\tau}^2\}$ and adds all components of the masked gradient (polynomial) to obtain $\overline{sum} = \sum_{j=1}^{\tau} \sum_{k=1}^{n_f} \bar{x}_{i,j,k}^2$, where $\bar{g}_{i,j}^2 = \{\bar{x}_{i,j,1}^2, \ldots, \bar{x}_{i,j,n_f}^2\}$ and $\bar{x}_{i,j,k}^2$ is the kth polynomial coefficient of $\bar{g}_{i,j}^2$. S_2 sends $\overline{sum}$ to S_1, who then obtains $sum = \overline{sum} - \sum_{j=1}^{\tau} \sum_{k=1}^{n_f} r_{i,j,k}$ through removing noises. Finally, if $sum = 1$, S_1 determines that g_i is normalized and accepts it; otherwise, S_1 discards the unnormalized g_i. Fig. 3 illustrates the detailed procedure of **ESecJudge**.

Authorized licensed use limited to: Anhui University. Downloaded on June 16,2026 at 03:35:52 UTC from IEEE Xplore. Restrictions apply.

For **ESecCos**, on inputting two normalized gradient ciphertexts $[\![g_a]\!] = \{[\![g_{a,1}]\!], \ldots, [\![g_{a,\tau}]\!]\}$ and $[\![g_b]\!] = \{[\![g_{b,1}]\!], \ldots, [\![g_{b,\tau}]\!]\}$ with $\| g_a \| = 1$ and $\| g_b \| = 1$, S_1 interacts with S_2 to calculate the cosine similarity of g_a and g_b as $\cos_{ab} = \frac{g_a \odot g_b}{\|g_a\| \times \|g_b\|} = g_{a,1} \times g_{b,1} + \cdots + g_{a,\tau} \times g_{b,\tau}$. In this ciphertext computation, the linear and non-linear operations correspond to additions and multiplications, respectively. We leverage S_1 to perform multiplication operations locally. The resulting computations are then masked and sent to S_2 for the addition operations.

In particular, first, S_1 performs the ciphertext multiplication and masking to obtain $[\![\bar{g}_{c,\Sigma}]\!] = \sum_{j=1}^{\tau} [\![g_{a,j} \times g_{b,j} + r_j]\!]$, where $\{r_j\}_{j\in[1,\tau]}$ are randomly selected noise polynomials by S_1. Then, S_2 obtains the plaintext $\bar{g}_{c,\Sigma} = \{x_{c,j,1}, \ldots, x_{c,j,n_f}\}$ by jointly decrypting $[\![\bar{g}_{c,\Sigma}]\!]$ with S_1 and sends the masked cosine similarity $\overline{\cos}_{ab} = \sum_{k=1}^{n_f} \sum_{j=1}^{\tau} x_{c,j,k}$ to S_1, where $x_{c,j,k} = \sum_{j=1}^{\tau} (g_{a,j,k} \times g_{b,j,k} + r_{j,k})$. Finally, S_1 obtains $\cos_{ab}$ through removing noises. A more specified **ESecCos** procedure is described in Fig. 4.

C. Construction of Our PBFL Scheme

In our enhanced scheme, a privacy-preserving defense mechanism is devised to protect against model poisoning attacks, leveraging new secure computation methods to ensure both privacy and computational correctness. Furthermore, we integrate this privacy-preserving defense with a newly proposed Byzantine-tolerant aggregation strategy, which is effective against both conventional and advanced adaptive poisoning attacks. Specifically, the enhanced scheme contains four processes detailed below.

1) *Setup:* In this process, KC generates public parameters/keys for the scheme and S_1 generates an initial model for the users. Specifically, given a security parameter ε', KC calls $\mathbf{FHE.Setup}$ and $\mathbf{FHE.KeyGen}$ to generate the public parameter PP and a tuple of key (sk, pk, evk). Then, KC calls $\mathbf{FHE.KeySplit}(sk)$ $n+1$ times to generate $n+1$ pairs of secret key shares (sk_1, sk_2) and $\{(sk_{s1_i}, sk_{s2_i}, sk_{u_i})\}_{i\in[1,n]}$. Finally, KC broadcasts $\{PP, pk, evk\}$, distributes $sk_1, \{sk_{s1_i}\}_{i\in[1,n]}$ to S_1, distributes $sk_2, \{sk_{s2_i}\}_{i\in[1,n]}$ to S_2, and distributes sk_{u_i} to U_i. S_1 selects a random initial model $W^{(0)}$ and sends $W^{(0)}$ to U_i.

2) *Local Training:* In each training iteration, each user trains its local model to obtain a local gradient, encrypts the local gradient, and sends the encrypted gradient to S_1. Specifically, in the tth training iteration, each user U_i trains its local model $W^{(t)}$ and flats its local gradient as a gradient vector $g_i^{(t)} \in \mathbb{R}^l$ whose length is l. The gradient vector is then normalized and uniformly divided into $\tau = \lceil l/n_f \rceil$ vectors $\{g_{i,j}^{(t)} \in \mathbb{R}^{n_f}\}_{j\in[1,\tau]}$ with length n_f. The elements 0 s are padded at the end of $g_i^{(t)}$ when less than n_f elements are left. Then, U_i encrypts each vector $g_{i,j}^{(t)}$ as $[\![g_{i,j}^{(t)}]\!]$ by calling $\mathbf{FHE.Enc}$. Finally, the beginning user U_i sends its encrypted gradient $[\![g_i^{(t)}]\!] = \{[\![g_{i,1}^{(t)}]\!], \ldots, [\![g_{i,\tau}^{(t)}]\!]\}$ to S_1 and the malicious user $U_i \in U^*$ may launch poisoning attacks and then send its encrypted poisonous gradients $[\![g_{i*}^{(t)}]\!] = \{[\![g_{i*,1}^{(t)}]\!], \ldots, [\![g_{i*,\tau}^{(t)}]\!]\}$ to S_1.

3) *Privacy-Preserving Defense:* In each training round, the two servers collaboratively execute the privacy-preserving defense logic. Specifically, S_1 first collects encrypted local gradients from all participating users and transmits the complete batch of masked ciphertexts to server S_2. Server S_2 then applies a random permutation to the received ciphertexts to obfuscate user identities prior to any joint computation (including normalization judgment and cosine similarity measurement). This permutation ensures that S_1 cannot associate any scalar output with a specific user. The privacy-preserving defense procedure consists of the following three secure subroutines:

- *Normalization Judgment:* The server S_1 determines whether the received encrypted local gradients $[\![g_i^{(t)}]\!] = \{[\![g_{i,1}^{(t)}]\!], \ldots, [\![g_{i,\tau}^{(t)}]\!]\}$ are regularized by invoking the verification function $\mathbf{ESecJudge}([\![g_i^{(t)}]\!])$. Only those passing the verification are considered *trusted* and included in the index set $\mathcal{I}_{\text{trusted}}$.
- *Poisonous Gradient Baseline Identification:* For all trusted gradients $\{[\![g_i^{(t)}]\!]\}_{i\in\mathcal{I}_{\text{trusted}}}$, the server computes the cosine similarity with the previously aggregated global gradient $[\![g_\Sigma^{(t-1)}]\!]$ as $\cos_{i\Sigma} \leftarrow \mathbf{ESecCos}([\![g_i^{(t)}]\!], [\![g_\Sigma^{(t-1)}]\!])$, for $i \in \mathcal{I}_{\text{trusted}}$. The gradient having the *lowest* similarity score is selected as the poisonous baseline $[\![g_*^{(t)}]\!]$.
- *Byzantine-Tolerant Aggregation:* To defend against adversarial behaviors while incentivizing honest participants, the server constructs a dual-scoring aggregation strategy using both confidence and credit scores.
  - *Confidence Computation:* For each user $i \in \mathcal{I}_{\text{trusted}}$, the cosine similarity between their local gradient and the poisonous baseline is computed as
  $$\cos_{i*} \leftarrow -\,\mathbf{ESecCos}([\![g_i^{(t)}]\!], [\![g_*^{(t)}]\!]).$$
  These similarity values are then normalized to obtain confidence scores:
  $$co_i \leftarrow \frac{\exp(\cos_{i*} - \max_j \cos_{j*})}{\sum_{j\in\mathcal{I}_{\text{trusted}}} \exp(\cos_{j*} - \max_k \cos_{k*})}.$$
  - *Credit Score Update:* Each participant!¯s credit score cs_i is updated with exponential smoothing
  $$cs_i \leftarrow \alpha \cdot cs_i + (1-\alpha) \cdot co_i,$$
  where $\alpha \in [0.7, 0.95]$ is a decay coefficient to balance historical consistency and current behavior.
  - *User Aggregation Weights:* The users' aggregation weights are computed based on the product of credit and confidence scores
  $$w_i \leftarrow \frac{cs_i \cdot co_i}{\sum_{j\in\mathcal{I}_{\text{trusted}}} cs_j \cdot co_j}.$$
  - *Adaptive User Filtering:* To further enhance robustness against adversarial manipulation and reduce the influence of anomalous or overconfident updates, the server performs an optional three-stage user filtering strategy before final aggregation:
    i) *Adaptive Weight Mixture*– The server first interpolates between uniform weighting and the learned

Authorized licensed use limited to: Anhui University. Downloaded on June 16,2026 at 03:35:52 UTC from IEEE Xplore. Restrictions apply.

robust weights based on the current round index t, using a decaying mixture coefficient:

$$\lambda \leftarrow \max\left(0, 1 - \frac{t - T_{\text{warmup}}}{T_{\text{total}} - T_{\text{warmup}}}\right),$$

$$\tilde{w}_i \leftarrow \lambda \cdot \frac{1}{n} + (1 - \lambda) \cdot w_i,$$

where $\tilde{w}_i$ denotes the adjusted aggregation weight for user i, and n is the total number of users. This allows the server to rely more on uniform weighting in early rounds and gradually transition to confidence-driven weighting as training progresses. Users with $\tilde{w}_i < \theta$ (e.g., $\theta = 0.05$) are excluded from the aggregation.

ii) *Weight-Based Outlier Suppression*– To mitigate the impact of disproportionately dominant updates, the server identifies the top-$\frac{n}{2}$ largest weights $\{w_i\}_{i \in \mathcal{I}_{\text{trusted}}}$, sorts them in descending order, and inspects adjacent weight differences. If a sharp drop is detected (i.e., $w_i - w_{i+1} > \delta$, with $\delta = \frac{1}{2n}$), users ranked above the drop are discarded. This ensures fairness and guards against potential gradient manipulation. The remaining users form the final participant set $\mathcal{I}_{\text{select}}$.

iii) *Credit Penalty for Filtered Users*– To enhance long-term robustness, users filtered out during either the *Normalization Judgment* or *Outlier Suppression* stages are penalized by applying differentiated scaling factors to their credit scores. Specifically, a multiplicative decay $\gamma_1 \in [0, 1]$ is applied to users excluded by normalization, while an inflation factor $\gamma_2 > 1$ is used for users removed during outlier suppression. This adjustment modifies each user's trust trajectory in future rounds accordingly.

Finally, the weighted aggregation of encrypted gradients is computed as

$$[\![g_\Sigma^{(t)}]\!] \leftarrow \sum_{i \in \mathcal{I}_{\text{select}}} \frac{w_i}{\sum_{j \in \mathcal{I}_{\text{select}}} w_j} \cdot [\![g_i^{(t)}]\!].$$

4) *Model Update:* Each user U_i receives the partial decryption shares $[g_\Sigma^{(t)}]_{s1_i}$ and $[g_\Sigma^{(t)}]_{s2_i}$, computed independently by servers S_1 and S_2 using $\mathbf{FHE.PartDec}$ with secret keys sk_{s1_i} and sk_{s2_i}, respectively:

$$[g_\Sigma^{(t)}]_{s1_i} \leftarrow \mathbf{FHE.PartDec}_{sk_{s1_i}}([\![g_\Sigma^{(t)}]\!]),$$

$$[g_\Sigma^{(t)}]_{s2_i} \leftarrow \mathbf{FHE.PartDec}_{sk_{s2_i}}([\![g_\Sigma^{(t)}]\!]).$$

Each user completes decryption using its private key sk_{u_i}

$$[g_\Sigma^{(t)}]_{u_i} \leftarrow \mathbf{FHE.PartDec}_{sk_{u_i}}([\![g_\Sigma^{(t)}]\!]),$$

$$g_\Sigma^{(t)} \leftarrow \mathbf{FHE.FullDec}([g_\Sigma^{(t)}]_{s1_i}, [g_\Sigma^{(t)}]_{s2_i}, [g_\Sigma^{(t)}]_{u_i}).$$

and updates its local model using

$$W^{(t+1)} \leftarrow W^{(t)} - \eta^{(t)} \cdot g_\Sigma^{(t)},$$

where $\eta^{(t)}$ is the learning rate of round t.

Our proposed privacy-preserving defense is designed to be robust against both conventional data poisoning attacks (as examined in [26]) and more sophisticated adaptive model poisoning attacks, including the AGR-tailored and AGR-agnostic strategies introduced in [53]. To counter these evolving threats, our defense integrates several key mechanisms that jointly enhance resilience:

- *Dual-Scoring Trust Assessment:* A two-tier scoring system is employed, combining per-round cosine-based *confidence scores* and long-term *credit scores*. This joint quantification reflects both short-term gradient consistency and historical user reliability, effectively preventing adaptive adversaries from persistently imitating benign behaviors to evade detection.
- *Exponential Credit Score Smoothing:* Credit scores are updated via exponential smoothing with decay factor $\alpha \in [0.7, 0.95]$, enabling the system to gradually penalize users with recurrent suspicious updates while remaining tolerant to occasional noise in benign gradients.
- *Soft Confidence Normalization:* Instead of applying hard thresholding, we utilize softmax-based normalization of negative cosine similarities to compute smooth confidence scores. This soft treatment maintains sensitivity to fine-grained deviations and provides robustness against gradient-level perturbations crafted by adaptive attackers.
- *Adaptive Weight Mixing and Thresholding:* An adaptive mixing scheme transitions from uniform weighting to fully trust-weighted aggregation as training progresses, enhancing early-round stability. Users with extremely low trust weights (below threshold θ) are excluded from aggregation to reduce stealth attack surfaces.
- *Outlier Suppression via Drop Detection:* Our outlier suppression step explicitly targets highly confident yet anomalously extreme updates, which are likely crafted to exploit the statistical patterns of benign gradients for malicious purposes. This mechanism is particularly critical for defending against adaptive adversaries, such as those employing AGR-tailored strategies. In such attacks, malicious gradients are deliberately constructed based on the distributional characteristics of benign updates, typically by adding a scaled deviation vector to their mean. These adversarial updates often evade baseline filters due to their close proximity to the benign gradient space. Furthermore, they may achieve abnormally high confidence scores by aligning even more closely with the directionality of benign updates than the benign users themselves.
- *Credit Penalty for Filtered Users:* This mechanism addresses the risk of adaptive adversaries regaining trust too easily after being filtered. Specifically, users removed during the *Normalization Judgment* step are presumed to have produced non-normalized or low-quality gradients and thus receive a credit decay to reduce their future influence. In contrast, users filtered via the *Outlier Suppression* mechanism may have presented gradients that appear highly confident yet abnormally extreme. These gradients often arise from adversarial strategies that construct malicious updates by scaling deviations from the benign gradient

Authorized licensed use limited to: Anhui University. Downloaded on June 16,2026 at 03:35:52 UTC from IEEE Xplore. Restrictions apply.

mean, making them closely aligned with benign directionality and capable of evading baseline checks. Since such users may initially appear trustworthy, a reverse penalty (i.e., increasing their credit score using $\gamma_2 > 1$) makes them more likely to be flagged again in subsequent outlier filtering, reinforcing detection consistency across rounds.

VI. Theoretical Analysis

In this section, we delve into an analysis of both the security and complexity of the enhanced scheme.

A. Security Analysis

We briefly discuss the security guarantees of our protocol under the defined adversarial model. Our PBFL framework protects the confidentiality of honest users' local gradients against a powerful adversary that may control any subset of users and at most one of the two servers, assuming they do not collude. Specifically, the protocol ensures that the honest users' gradients remain computationally indistinguishable from random to the adversary and the global model is returned in encrypted form and can only be decrypted by users. We formalize these guarantees using a simulation-based security definition, and provide a rigorous proof in Appendix C. The proof constructs an ideal functionality capturing the defense logic and shows that the real protocol securely realizes this functionality via a series of indistinguishable hybrid executions.

B. Convergence Analysis

We provide a detailed theoretical convergence analysis of the proposed PBFL scheme in Appendix D. Specifically, the aggregation error is decomposed into three components: (i) residual Byzantine bias, caused by the influence of malicious updates; (ii) stochastic gradient noise, stemming from the randomness in local sampling; and (iii) gradient diversity, resulting from data heterogeneity across users. We then formally show that, under standard smoothness and bounded-variance assumptions, the proposed algorithm achieves the same sub-linear convergence rate $\mathcal{O}(1/\sqrt{T})$ as classical FedAvg, up to an additive error term that is proportional to the Byzantine fraction β and decreases as the number of trusted participants increases. This result theoretically confirms the robustness and efficiency of our defense mechanism.

C. Performance Analysis

We provide a detailed performance comparison of computational and communication complexities between our proposed scheme and ShieldFL, which is presented in Appendix E. The analysis covers all major phases including *Setup*, *Local Training*, *Privacy-Preserving Defense*, and *Model Update*. Our results show that, both schemes have communication and computational complexity of $O(n)$ and due to ciphertext computation simplicity and cryptographic primitive efficiency, our scheme significantly outperforms ShieldFL in both computation and communication efficiency. Specifically:

- *Computational Complexity:* Our scheme reduces asymptotic encryption, decryption, and homomorphic operation costs by a factor related to the smaller ciphertext dimension ($n_f q \ll N^2$). This translates to a lower overhead in all training rounds.
- *Communication Cost:* Since homomorphic multiplication in our secure computation methods $\mathbf{ESecCos}$ and $\mathbf{ESecJudge}$ is executed locally without inter-party interaction, its communication cost is effectively zero. In contrast, ShieldFL incurs significant overhead due to interactive homomorphic multiplication in their secure computation protocols.

This theoretical performance advantage is also reflected in our experimental results (Section VII-B), validating the practical efficiency of our scheme.

D. Discussion and Practical Considerations

While our enhanced scheme achieves significant improvements in privacy and efficiency, several practical considerations remain for real-world FL deployments. This section discusses two key aspects: the feasibility of decentralizing the Key Center and the challenges of defending against adaptive and malicious adversaries.

1) Decentralizing the Key Center: Our current design assumes a trusted centralized Key Center for key management and distribution. This assumption, however, may not always hold in heterogeneous or cross-silo FL environments where participants belong to different administrative domains with varying trust levels. To mitigate this limitation, decentralized approaches such as threshold cryptography and Distributed Key Generation (DKG) protocols can be adopted. These techniques distribute key management responsibilities among multiple independent parties, thereby eliminating the single point of trust and reducing the risk of key compromise. In addition, blockchain-based solutions and secure multiparty computation (MPC) protocols could be integrated to provide transparent, auditable, and tamper-resistant key management. While these decentralized mechanisms can enhance robustness and trustworthiness, they inevitably introduce challenges such as increased communication overhead and coordination complexity, which require further study.

2) Defending Against Adaptive and Malicious Threats: Our security analysis primarily considers malicious users and at most one corrupted (malicious) server, ensuring the privacy of honest users' data under the assumption that the two servers do not collude. However, in real-world FL deployments, adversaries may behave both maliciously and adaptively, dynamically deviating from the protocol to launch sophisticated attacks such as adaptive model poisoning or adaptive inference.

We explicitly evaluate our framework under an *adaptive model poisoning* threat, where the adversary dynamically adjusts its malicious gradients based on the gradients of benign users before injecting them into the training process. This adaptive strategy makes detection substantially more difficult. Our experimental results demonstrate that the proposed framework can effectively mitigate this class of adaptive attacks. Beyond model poisoning, more advanced adaptive threats have been reported, including adaptive backdoor attacks that iteratively modify backdoor triggers to evade defenses and adaptive inference attacks that progressively extract private information from

Authorized licensed use limited to: Anhui University. Downloaded on June 16,2026 at 03:35:52 UTC from IEEE Xplore. Restrictions apply.

model updates. These attacks evolve over training rounds and present challenges that our current design does not fully address.

Future work will focus on strengthening the framework against these stronger adversarial models by extending robustness beyond poisoning to cover adaptive backdoor and inference-based attacks. Potential directions include developing adaptive anomaly detection frameworks that dynamically adjust detection thresholds or defense strategies based on adversarial behaviors observed during training (e.g., online learning-based detectors that evolve as attacks change) and incorporating temporal consistency checks and cross-round validation mechanisms to detect adversaries that gradually adapt their malicious updates to evade static defenses. Furthermore, leveraging ensemble or diversity-based defenses (where multiple defense mechanisms operate in parallel and dynamically reconfigure based on the detected threat level) could make it substantially harder for adaptive attackers to circumvent all defenses simultaneously. Beyond adaptive attacks, we also plan to address the threat of malicious servers that compromise computation correctness by integrating verifiable computation or zero-knowledge proofs to guarantee both privacy and correctness even when a server is malicious.

VII. Experimental Evaluation

In our evaluation, we focus on the three perspectives: (1) We validate our privacy attacks on ShieldFL. (2) We evaluate the defense effect of our enhanced scheme against different model poisoning attacks (including targeted and untargeted attacks). (3) We compare the communication traffic and the computational cost of our enhanced scheme with the SOTA schemes.

A. Experimental Setup

We conduct the experiments on a computer with Intel Core i7-6700 CPU. The Homomorphic encryptions are coded with C++ language, and the FL model is coded with Python 3.9.13 under the PyTorch framework. The gradients of the model are encrypted by calling the dynamic library we generated for the homomorphic encryption codes.

Dataset, Model, and Parameter: In order to facilitate the visual display of subsequent experimental results, we use the benchmark image datasets MNIST [54] and CIFAR10 [55] as the experimental datasets and construct IID and non-IID datasets. The MNIST dataset consists of images for handwritten digits of ten classes (from '0' to '9'). The digit in each training example has been size-normalized and centered in a 28×28 size gray-scale image. The MNIST dataset has 60,000 training examples and 10,000 test examples. The CIFAR10 dataset contains 60,000 many 32×32 color images in ten classes (cat, car, etc.), with 6,000 images per class, and it has 50,000 training examples and 10,000 test examples. There are ten users in our FL model. For IID setting, we uniformly partition the training dataset among the ten users. For non-IID setting, we use Dirichlet distribution with $\alpha = 0.5$ [56]. In the experiments, we consider convolutional neural networks LeNet-5 [57] with MNIST dataset and ConvNet [58] with CIFAR10 dataset. In training, we utilize SGD optimizer and set batch size and learning rate to 32 and 0.01, respectively. The precision of the two models is 32 bits. In our Byzantine-tolerant aggregation framework, we employ an exponential smoothing factor of $\alpha = 0.9$ and set the minimum weight threshold to $\theta = 0.01$. To suppress dominant adversarial updates, we integrate a weight-based outlier detection scheme with drop threshold $\delta = 0.05$, alongside a credit penalty mechanism with $\gamma_2 = 1, \gamma_2 = 1.5$. We set the security level of HE algorithms to 128 bits. The ciphertext moduli of ShieldFL and our scheme are $N^2 = 2^{3072\times 2}$ and $q = 2^{56}$, respectively. The polynomial modulus degree of our scheme is $n_f = 2048$.

Privacy Attack Methods: Our proposed privacy attacks expose the information of users' local gradients. Specifically, the correlations among arbitrary gradients are leaked, and in some cases, the gradients are directly disclosed in plaintext. To demonstrate the effectiveness of our attacks, we employ the existing GS model inversion method [2] to reconstruct raw training data from the exposed gradients. In the GS method, cosine similarity between users' local gradients and falsified gradients is used to optimize the reconstruction of FL data.

Model Poisoning Attack Methods: Model poisoning refers to any manipulation, whether through training data or model updates, that leads to performance degradation or targeted misclassification in the global model. In our experiments, model poisoning attacks include data-based poisoning and direct model manipulation described as follows.

- *Data-Based Model Poisoning (via poisoned local dataset)* These attacks corrupt the training data of malicious users, indirectly affecting their model updates.
  - *Label Flipping [15]:* The label flipping attack aims to manipulate the model's predictions in a specific direction, towards a predefined target class or outcome. To simulate this attack, we relabel malicious users' training samples of the source class "a" as the target class "b" and then use these samples to train poisonous gradients.
  - *Random Perturbation [26]:* The random perturbation attack seeks to perturb the input data in a way that induces misclassification or errors in the model's predictions, without specifying a particular target class. To simulate this attack, we randomly relabel the training samples of malicious users as error classes, generating poisonous gradients.
- *Direct Model Poisoning (via gradient manipulation):* We adopt the advanced adaptive attack strategy proposed by Shejwalkar et al. [53], which directly poisons the model by crafting malicious updates. In this setting, adversaries are assumed to have full knowledge of benign users' updates and exploit this information to tailor their malicious gradients accordingly. The adaptive attacks can be broadly categorized into two types:
  - *AGR Tailored Attack:* The adversary leverages the gradients of benign users to compute a reference aggregate, and then crafts malicious updates by adding optimized perturbations. These perturbations are carefully designed to both resemble benign updates and maximize their adversarial effect, ensuring they are selected during

Authorized licensed use limited to: Anhui University. Downloaded on June 16,2026 at 03:35:52 UTC from IEEE Xplore. Restrictions apply.

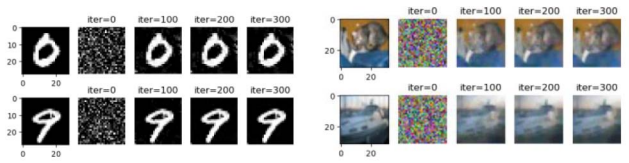
aggregation. The attack is customized to exploit the specific aggregation rule (AGR) used by the server, allowing it to bypass detection and maximize impact.

– *AGR Agnostic Attack:* Unlike AGR-tailored attacks, this method does not assume knowledge of the server's aggregation rule. The adversary crafts malicious updates within a small neighborhood of benign gradients, ensuring they closely resemble benign updates to evade detection, while still degrading the global model. This attack remains effective even when the aggregation algorithm is unknown.

We denote the proportion of malicious users to the total number of users as the attack ratio, denoted by Att_{ro}, in the context of model poisoning attacks. In our experiments, we evaluate our defense under a challenging setting with $Att_{ro} = 50\%$, which represents the theoretical upper bound for most robust aggregation methods. For adaptive attacks, we consider a more practical scenario with $Att_{ro} = 20\%$, a commonly adopted setting in robust FL literature that facilitates fair and consistent comparison across different defense strategies.

Baselines: To comprehensively evaluate the performance of our proposed scheme, we compare it against three categories of baselines:

- *Standard FL (Plaintext, No Defense):*
  – *FedSGD [33]:* Performs standard plaint FL without any defense against privacy leakage or model poisoning attacks. This baseline serves as a performance upper bound in benign settings.
- *Robust FL (Robustness Only):*
  – *Krum [14]:* Calculates the sum of squared distances between each user's gradient and its nearest neighbors, and then selects the gradient that is closest to the majority of gradients, minimizing the influence of malicious updates.
  – *FoolsGold [59]:* Leverages the similarity of user updates to identify and eliminate malicious updates. During each update, it dynamically adjusts the learning rate based on the similarity of user contributions, reducing the impact of malicious users on the global model.
  – *FLTrust [15]:* Uses a root dataset to establish trust, applies normalization to align the magnitudes of user updates with the server model, and aggregates the updates using trust scores to ensure robustness against Byzantine attacks.
  – *DnC [53]:* Applies principal component analysis (PCA) to identify the dominant direction of variation among the submitted updates. Each update is then projected onto this principal direction, and those with the largest projection values, i.e., the most deviating updates along the principal axis, are filtered out.
  – *FLARE [60]:* Uses maximum mean discrepancy (MMD) to compute the discrepancy in the penultimate layer representations (PLR) between each pair of model updates, thereby estimating the trust score for each model. Updates with higher trust scores are assigned greater weights, which reduces the impact of malicious updates on the global model.



(a) GS with MNIST dataset (b) GS with CIFAR10 dataset

Fig. 5. The visualization showing the model inversion from gradients to raw data of ShieldFL.

  – *FLAME [61]:* Utilizes dynamic clustering to detect and exclude malicious models based on their angular deviations from the majority of updates. The models with the largest deviations are considered malicious and removed. Additionally, it applies weight clipping during aggregation to reduce the influence of these malicious models, and further injects Gaussian noise to ensure the global model is not compromised by backdoor attacks.

  These methods enhance aggregation robustness against model poisoning attacks under plaintext settings, but do not provide cryptographic privacy guarantees.
- *Privacy-Preserving and Robust FL (Robust + Private):*
  – *RSFL [20]:* Achieves robustness and privacy by leveraging multi-key CKKS homomorphic encryption [34] and Euclidean-distance-based outlier filtering.
  – *PILE [41]:* Uses a threshold Paillier cryptosystem [62], zero-knowledge proofs, and commitment mechanisms to guarantee gradient confidentiality and integrity.
  – *ShieldFL [26]:* The original version of our proposed framework, which designs the privacy-preserving Byzantine defense mechanism using the two-trapdoor Paillier cyrtosystem and the cosine similarity.

  These schemes operate under encrypted gradients, where individual local updates are not observable by either the server or malicious users.

B. Experimental Results

Privacy attack Effect on ShieldFL: Fig. 5 shows the experimental results of model inversion on ShieldFL. The figure shows that the exposed gradients from ShieldFL can recover the raw data of all classes in just a few training iterations (100 training rounds). This experimental result validates the effectiveness of our attacks on ShieldFL.

Defense against Model Poisoning Attacks: Fig. 6 presents the accuracy trajectories of our defense scheme under four representative model poisoning attacks on CIFAR10. For each attack, we compare the test accuracy of our defense against two baselines: (1) an ideal FL training with no adversaries, and (2) a standard FL scheme attacked without defense. Results are reported under both IID and non-IID user data distributions. Across all attack types, we observe that poisoning attacks severely degrade the performance of the vanilla FL system, particularly in the non-IID setting. However, our proposed scheme substantially mitigates the accuracy drop in both settings. Under label flipping (Fig. 6(a)) and random perturbation (Fig. 6(b)), our defense

Authorized licensed use limited to: Anhui University. Downloaded on June 16,2026 at 03:35:52 UTC from IEEE Xplore. Restrictions apply.

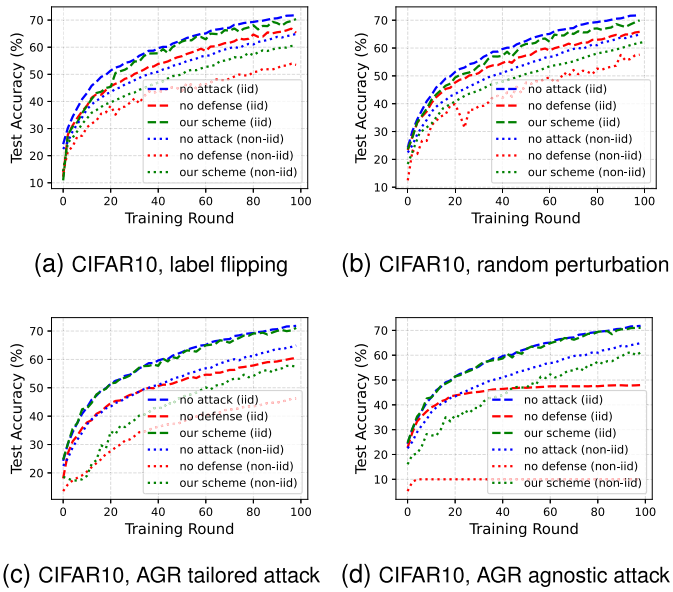


Fig. 6. Accuracy analysis of our scheme.

TABLE III
ACCURACY COMPARISON BETWEEN OUR SCHEME AND EXISTING PBFL SCHEMES

Setting	Attack	Krum	FLTrust	RSFL	PILE	ShieldFL	Ours
IID	LPA	90.7%	75.1%	81.2%	95.8%	90.3%	**96.3%**
	RPA	90.4%	69.7%	80.9%	92.7%	90.2%	**94.5%**
Non-IID	LPA	71.4%	37.8%	79.8%	89.8%	90.1%	**95.2%**
	RPA	78.3%	43.4%	80.3%	89.5%	89.4%	**93.4%**

Notes. MNIST dataset, 100 training rounds, $Att_{ro} = 50\%$.

closes the performance gap to the no-attack upper bound, recovering $85\% - 95\%$ of the clean accuracy. For AGR-tailored (Fig. 6(c)) and AGR-agnostic (Fig. 6(d)) adaptive attacks, which craft updates to bypass standard defenses, our method achieves strong defense performance, consistently outperforming the no-defense baseline by large margins. These results validate that our scheme is broadly effective across attack types, attack intensities, and data heterogeneity levels, consistently pushing the performance back toward the no-attack ideal.

Table III presents a comparison of our scheme with state-of-the-art PBFL defense mechanisms against model poisoning attacks, including label flipping attack (LFA) and random perturbation attack (RPA). Since all the PBFL schemes utilize privacy-preserving techniques to protect users' local gradients, the adversary cannot access the benign users' gradients and, therefore, cannot launch the adaptive attacks previously discussed. As a result, we have not tested the defense effectiveness of these PBFL schemes against adaptive attacks. The table clearly shows that our scheme outperforms the comparison schemes in defending against both label flipping and random perturbation attacks, regardless of whether the data is IID or non-IID.

We further evaluate the resilience of our robust-aggregation method under adaptive poisoning attacks, where adversaries are assumed to have full knowledge of benign users' updates and attempt to craft malicious updates accordingly. Specifically, attackers estimate the direction of the benign gradient and construct poisoned updates in the form of $\Delta x + z \cdot w$, where w is adaptively chosen to evade detection. In this setting, we only compare our scheme with existing *robust federated learning* baselines (i.e., Krum, FLTrust, FoolsGold, DnC, FLARE, and FLAME), as these schemes operate in plaintext, allowing adversaries to observe the benign updates. This transparency enables the attacker to effectively manipulate the gradients based on the observed direction. Table IV presents the evaluation results of various robust aggregation methods under adaptive poisoning attacks. The metric I_θ represents the accuracy drop caused by the attack on a benign model, with smaller values indicating stronger robustness. Our method consistently achieves the lowest or near-lowest I_θ across both IID and Non-IID data settings. Specifically, under the AGR-tailored attack in the IID setting, FLAME achieves $I_\theta = 0.64$, while our method achieves a comparable result of 0.67. Under the AGR-agnostic attack, our scheme outperforms all baselines, with the lowest accuracy degradation of 0.27, compared to 0.58 by FLAME and 0.82 by FLARE. In the more challenging Non-IID setting, some baseline methods fail to converge (N/C) or do not report results. Notably, our method remains robust, achieving $I_\theta = 0.31$ under AGR-tailored and 0.49 under AGR-agnostic attacks, outperforming FLARE. These results demonstrate the strong defense capability and stability of our scheme across diverse threat settings.

Communication Traffic: Table V compares our scheme with the PBFL schemes RSFL, PILE, and ShieldFL on ciphertext number, ciphertext size, and network traffic on one training iteration. The gradient dimensions of LeNet-5 and ConvNet are 61.69 K and 2.90 M, respectively. Compared to the comparative schemes, our scheme has the fewest number of ciphertexts, the smallest ciphertext size, and the lowest communication traffic. In particular, the ciphertext number in a gradient of our scheme is $10\times$ less than that of RSFL and is about $2000\times$ less than that of PILE and the original ShieldFL. This is true because the encryption algorithm of our scheme can encrypt $n_f = 2048$ plaintexts into a ciphertext at once while the encryption algorithm of ShieldFL/PILE can only encrypt plaintext one by one. The existing Single Instruction Multiple Data (SIMD) technology cannot be used in ShieldFL to pack multiple plaintexts into one ciphertext, or else the secure computation methods including $\mathbf{SecJudge}$ and $\mathbf{SecCos}$ cannot be correctly implemented. The ciphertext size and the network traffic are linearly correlated to the ciphertext number and modulus. From the table, we know that the ciphertext size of our scheme is about $5\times, 70\times$, and $23\times$ smaller than that of RSFL, PILE, and ShieldFL, respectively. And the network traffic among ten users on transmitting ciphertexts of our scheme is approximately $9\times, 739\times$ and $245\times$ smaller than that of RSFL, PILE, and ShieldFL, respectively.

Computational Cost. Fig. 7 compares the experiment results of the running time of our scheme and the PBFL schemes RSFL, PILE, and ShieldFL. The experiments are run for 100 training rounds, and their averaged results are drawn in the figure. From Fig. 7, we see that our scheme reduces the computational efficiency of RSFL by more than $3\times$. Besides, compared with PILE and ShieldFL, our scheme improves the computational efficiency of the privacy-preserving defense process by about

Authorized licensed use limited to: Anhui University. Downloaded on June 16,2026 at 03:35:52 UTC from IEEE Xplore. Restrictions apply.

TABLE IV
IMPACT OF ADAPTIVE ATTACKS ON MODEL ACCURACY (I_θ) ACROSS OUR SCHEME AND EXISTING ROBUST FL SCHEMES

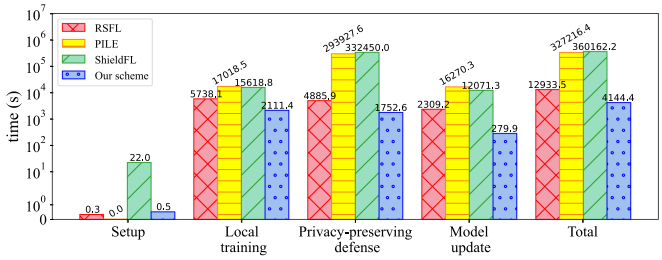
Setting	Attack	Krum	FLTrust	FoolsGold	DnC	FLARE	FLAME	Ours
IID	AGR tailored	33.9	1.7	2.2	1.8	1.64	**0.64**	0.67
	AGR agnostic	28.5	-	1.9	1.2	0.82	0.58	**0.27**
Non-IID	AGR tailored	41.5	6.9	14.41	2.5	N/C	-	**0.31**
	AGR agnostic	31.7	2.0	-	1.8	2.74	-	**0.49**

Notes. MNIST dataset, 100 training rounds, $Att_{ro} = 20\%$.
"N/C": Model fails to converge under attack (test accuracy $\approx 10\%$).
"-": Evaluation under this setting has not been reported.

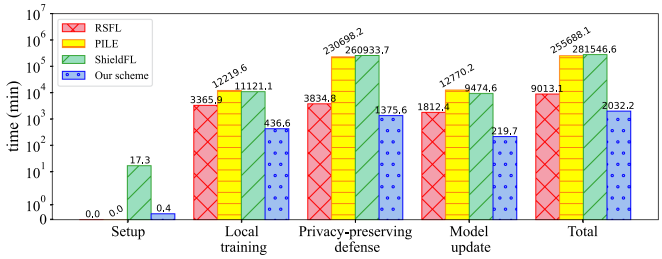
TABLE V
COMPARISONS OF CIPHERTEXT NUMBER/SIZE AND NETWORK TRAFFIC ON ONE ITERATION

Scheme	Item	LeNet-5	ConvNet
FedSGD	Gradient dimension	61.69K	2.90M
	$Gradient	0.24MB	11.08MB
RSFL	#Ciphertext in a gradient	341	15.24K
	$Ciphertext of a gradient	2.2MB	0.44GB
	Network traffic	0.74GB	5.17GB
PILE	#Ciphertext in a gradient	61.69K	2.90M
	$Ciphertext of a gradient	29.49MB	6.24GB
	Network traffic	8.59GB	406.8GB
ShieldFL	#Ciphertext in a gradient	61.69K	2.90M
	$Ciphertext of a gradient	9.83MB	2.08GB
	Network traffic	2.87GB	135.06GB
Our scheme	#Ciphertext in a gradient	**31**	**1419**
	$Ciphertext of a gradient	**0.42MB**	**0.04GB**
	Network traffic	**5.50MB**	**0.55GB**

Notes. "#A" represents "the number of A".
"$A" represents "the size of A".



(a) Running time for training LeNet-5



(b) Running time for training ConvNet

Fig. 7. Comparisons of running time.

two orders of magnitude for both LeNet-5 and ConvNet models. Our scheme achieves a total running time that is more than $80\times$ faster than that of PILE and ShieldFL for both LeNet-5 and ConvNet models.

VIII. CONCLUSION

In this paper, we identify security weaknesses in privacy-preserving and Byzantine-robust federated learning (PBFL) schemes with the two-server model and proposes an enhanced PBFL solution. In our enhanced scheme, a Byzantine-tolerant aggregation method is devised to fortify against model poisoning attacks. Additionally, we develop the enhanced secure normalization judgment method and secure cosine similarity measurement method to protect the privacy of the defense process while ensuring the correctness of the computations. Our scheme guarantees privacy preservation and resilience against model poisoning attacks, even in scenarios with heterogeneous datasets that are non-IID (Independently Identically Distributed). Moreover, it demonstrates superior efficiency in computational and communication performance compared to the existing state-of-the-art PBFL schemes.

REFERENCES

[1] B. Hitaj, G. Ateniese, and F. Perez-Cruz, "Deep models under the GAN: Information leakage from collaborative deep learning," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2017, pp. 603–618.
[2] J. Geiping, H. Bauermeister, H. Dröge, and M. Moeller, "Inverting gradients-how easy is it to break privacy in federated learning?," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, vol. 33, 2020, pp. 16937–16 947.
[3] H. Yang et al., "Fast generation-based gradient leakage attacks: An approach to generate training data directly from the gradient," *IEEE Trans. Depend. Sec. Comput*, vol. 22, no. 1, pp. 132–145, Jan./Feb. 2025, doi: 10.1109/TDSC.2024.3387570.
[4] A. N. Bhagoji, S. Chakraborty, P. Mittal, and S. Calo, "Analyzing federated learning through an adversarial lens," in *Proc. Int. Conf. Mach. Learn.*, 2019, pp. 634–643.
[5] C. Shi et al., "Towards practical backdoor attacks on federated learning systems," *IEEE Trans. Depend. Sec. Comput.*, vol. 21, no. 6, pp. 5431–5447, Nov./Dec. 2024, doi: 10.1109/TDSC.2024.3376790.
[6] H. Li, X. Sun, and Z. Zheng, "Learning to attack federated learning: A model-based reinforcement learning attack framework," in *Proc. Neural Inf. Process. Syst.*, 2022, pp. 35007–35020.
[7] "Pets prize challenge: Phase 2 (financial crime)," 2023. [Online]. Available: https://www.drivendata.org/competitions/105/nistfederated-learning-2-financial-crime-federated/
[8] "Pets prize challenge: Phase 2 (pandemic forecasting-federated)," 2023. [Online]. Available: https://www.drivendata.org/competitions/103/nist-federated-learning-2-pandemic-forecasting-federated/
[9] L. Chen, X. Ding, Z. Bao, P. Zhou, and H. Jin, "Differentially private federated learning on non-IID data: Convergence analysis and adaptive optimization," *IEEE Trans. Knowl. Data Eng.*, vol. 36, no. 9, pp. 4567–4581, Sep. 2024, doi: 10.1109/TKDE.2024.3379001.
[10] C. Hu and B. Li, "MaskCrypt: Federated learning with selective homomorphic encryption," *IEEE Trans. Depend. Sec. Comput.*, vol. 22, no. 1, pp. 221–233, Jan./Feb. 2024, doi: 10.1109/TDSC.2024.3392424.
[11] D. Ye, S. Shen, T. Zhu, B. Liu, and W. Zhou, "One parameter defense: Defending against data inference attacks via differential privacy," *IEEE Trans. Inf. Forensics Secur.*, vol. 17, pp. 1466–1480, 2022.

Authorized licensed use limited to: Anhui University. Downloaded on June 16,2026 at 03:35:52 UTC from IEEE Xplore. Restrictions apply.

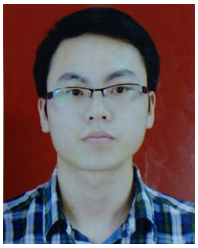
[12] Y. Chang, K. Zhang, J. Gong, and H. Qian, "Privacy-preserving federated learning via functional encryption, revisited," *IEEE Trans. Inf. Forensics Secur.*, vol. 18, pp. 1855–1869, 2023.
[13] Y. Zheng, S. Lai, Y. Liu, X. Yuan, X. Yi, and C. Wang, "Aggregation service for federated learning: An efficient, secure, and more resilient realization," *IEEE Trans. Depend. Sec. Comput*, vol. 20, no. 2, pp. 988–1001, Mar./Apr. 2023.
[14] P. Blanchard, E. M. E. Mhamdi, R. Guerraoui, and J. Stainer, "Machine learning with adversaries: Byzantine tolerant gradient descent," in *Proc. Neural Inf. Process. Syst.*, 2017, pp. 118–128.
[15] X. Cao, M. Fang, J. Liu, and N. Z. Gong, "FLTrust: Byzantine-robust federated learning via trust bootstrapping," in *Proc. Netw. Distrib. Syst. Secur. Symp.*, 2021. [Online]. Available: https://www.ndss-symposium.org/wp-content/uploads/ndss2021_6C-2_24434_paper.pdf
[16] T. Krauß, J. König, A. Dmitrienko, and C. Kanzow, "Automatic adversarial adaption for stealthy poisoning attacks in federated learning," in *Proc. Netw. Distrib. Syst. Secur. Symp.*, 2024. [Online]. Available: https://www.ndss-symposium.org/wp-content/uploads/2024-1366-paper.pdf
[17] H. Yan et al., "Recess vaccine for federated learning: Proactive defense against model poisoning attacks," in *Proc. Neural Inf. Process. Syst.*, 2024, pp. 8702–8713.
[18] J. Lu et al., "Depriving the survival space of adversaries against poisoned gradients in federated learning," *IEEE Trans. Inf. Forensics Secur.*, vol. 19, pp. 5405–5418, 2024.
[19] Y. Lin et al., "PPBR-FL: A privacy-preserving and byzantine-robust federated learning system," in *Proc. Int. Conf. Knowl. Sci. Eng. Manage.*, 2022, pp. 39–50.
[20] X. Hao, C. Lin, W. Dong, X. Huang, and H. Xiong, "Robust and secure federated learning against hybrid attacks: A generic architecture," *IEEE Trans. Inf. Forensics Secur.*, vol. 19, pp. 1576–1588, 2024.
[21] Z. Ma, J. Ma, Y. Miao, X. Liu, K.-K. R. Choo, and R. H. Deng, "Pocket Diagnosis: Secure federated learning against poisoning attack in the cloud," *IEEE Trans. Serv. Comput.*, vol. 15, no. 6, pp. 3429–3442, Nov./Dec. 2022.
[22] H. Kasyap, A. Manna, and S. Tripathy, "An efficient blockchain assisted reputation aware decentralized federated learning framework," *IEEE Trans. Netw. Serv. Manage.*, vol. 20, no. 3, pp. 2771–2782, Sep. 2023.
[23] R. Xu and Y. Chen, "μDFL: A secure microchained decentralized federated learning fabric atop IoT networks," *IEEE Trans. Netw. Serv. Manage.*, vol. 19, no. 3, pp. 2677–2688, Sep. 2022.
[24] M. Abdel-Basset, N. Moustafa, and H. Hawash, "Privacy-preserved cyberattack detection in industrial edge of things: A blockchain-orchestrated federated learning approach," *IEEE Trans. Ind. Informat*, vol. 18, no. 11, pp. 7920–7934, Nov. 2022.
[25] X. Liu, H. Li, G. Xu, Z. Chen, X. Huang, and R. Lu, "Privacy-enhanced federated learning against poisoning adversaries," *IEEE Trans. Inf. Forensics Secur.*, vol. 16, pp. 4574–4588, 2021.
[26] Z. Ma and J. Ma, "ShieldFL: Mitigating model poisoning attacks in privacy-preserving federated learning," *IEEE Trans. Inf. Forensics Secur.*, vol. 17, pp. 1639–1654, 2022.
[27] Z. Zhang et al., "LSFL: A lightweight and secure federated learning scheme for edge computing," *IEEE Trans. Inf. Forensics Secur.*, vol. 18, pp. 365–379, 2023.
[28] Y. Miao, Z. Liu, H. Li, K.-K. R. Choo, and R. H. Deng, "Privacy-preserving byzantine-robust federated learning via blockchain systems," *IEEE Trans. Inf. Forensics Secur.*, vol. 17, pp. 2848–2861, 2022.
[29] Y. Miao et al., "RFed: Robustness-enhanced privacy-preserving federated learning against poisoning attack," *IEEE Trans. Inf. Forensics Secur.*, vol. 19, pp. 5814–5827, 2024.
[30] R. Xu, S. Gao, C. Li, J. Joshi, and J. Li, "Dual Defense: Enhancing privacy and mitigating poisoning attacks in federated learning," in *Proc. Int. conf. Neural Inf. Process. Syst.*, 2024, pp. 70476–70498.
[31] A. Yazdinejad, A. Dehghantanha, H. Karimipour, G. Srivastava, and R. M. Parizi, "A robust privacy-preserving federated learning model against model poisoning attacks," *IEEE Trans. Inf. Forensics Secur.*, vol. 19, pp. 6693–6708, 2024.
[32] P. Tang, X. Zhu, W. Qiu, Z. Huang, Z. Mu, and S. Li, "FLAD: Byzantine-robust federated learning based on gradient feature anomaly detection," *IEEE Trans. Depend. Sec. Comput*, vol. 22, no. 4, pp. 3993–4009, Jul./Aug. 2025.
[33] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Proc. Int. Conf. Artif. Intell. Statst.*, 2017, pp. 1273–1282.
[34] H. Chen, W. Dai, M. Kim, and Y. Song, "Efficient multi-key homomorphic encryption with packed ciphertexts with application to oblivious neural network inference," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2019, pp. 395–412.
[35] J. Wu, W. Zhang, and F. Luo, "ESAFL: Efficient secure additively homomorphic encryption for Cross-Silo federated learning," *IEEE Trans. Depend. Sec. Comput.*, vol. 22, no. 1, pp. 3868–3884, Jul./Aug. 2025.
[36] J. Ma, S.-A. Naas, S. Sigg, and X. Lyu, "Privacy-preserving federated learning based on multi-key homomorphic encryption," *Int. J. Intell. Syst.*, vol. 37, no. 9, pp. 5880–5901, 2022.
[37] J. Wu, T. Sun, F. Luo, H. Wang, and W. Zhang, "Secure multi-key homomorphic encryption with application to privacy-preserving federated learning," 2025, *arXiv:2506.20101*.
[38] T. Kim, H. Kwak, D. Lee, J. Seo, and Y. Song, "Asymptotically faster multi-key homomorphic encryption from homomorphic gadget decomposition," in *Proc. ACM Conf. Comput. Commun. Secur.*, 2023, pp. 726–740.
[39] J. H. Cheon, A. Kim, M. Kim, and Y. Song, "Homomorphic encryption for arithmetic of approximate numbers," in *Proc. Int. Conf. Theory Appl. Cryptology Inf. Secur.*, 2017, pp. 409–437.
[40] M. Rathee, C. Shen, S. Wagh, and R. A. Popa, "ELSA: Secure aggregation for federated learning with malicious actors," in *Proc. IEEE Symp. Secur. Privacy*, 2023, pp. 1961–1979.
[41] X. Tang, M. Shen, Q. Li, L. Zhu, T. Xue, and Q. Qu, "PILE: Robust privacy-preserving federated learning via verifiable perturbations," *IEEE Trans. Depend. Sec. Comput*, vol. 20, no. 6, pp. 5005–5023, Nov./Dec. 2023.
[42] A. Roy Chowdhury, C. Guo, S. Jha, and L. van der Maaten, "EIFFeL: Ensuring integrity for federated learning," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2022, pp. 2535–2549.
[43] Z. Zhang, J. Li, S. Yu, and C. Makaya, "SAFELearning: Secure aggregation in federated learning with backdoor detectability," *IEEE Trans. Inf. Forensics Secur.*, vol. 18, pp. 3289–3304, 2023.
[44] T. Gehlhar, F. Marx, T. Schneider, A. Suresh, T. Wehrle, and H. Yalame, "SafeFL: MPC-friendly framework for private and robust federated learning," in *Proc. IEEE Symp. Secur. Privacy*, 2023, pp. 69–76.
[45] P. Mai, R. Yan, and Y. Pang, "RFLPA: A robust federated learning framework against poisoning attacks with secure aggregation," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2024, pp. 104329–104356.
[46] X. Li, X. Yang, Z. Zhou, and R. Lu, "Efficiently achieving privacy preservation and poisoning attack resistance in federated learning," *IEEE Trans. Inf. Forensics Secur.*, vol. 19, pp. 4358–4373, 2024.
[47] F. Mo, H. Haddadi, K. Katevas, E. Marin, D. Perino, and N. Kourtellis, "PPFL: Privacy-preserving federated learning with trusted execution environments," in *Proc. Int. Conf. Mobile Syst. Appl. Serv.*, 2021, pp. 94–108.
[48] A. Mondal, Y. More, R. H. Rooparaghunath, and D. Gupta, "Poster: FLATEE: Federated learning across trusted execution environments," in *Proc. IEEE Eur. Symp. Secur. Privacy*, 2021, pp. 707–709.
[49] S. Queyrut, V. Schiavoni, and P. Felber, "Mitigating adversarial attacks in federated learning with trusted execution environments," in *Proc. Int. Conf. Distrib. Comput. Syst.*, 2023, pp. 626–637.
[50] L. Braun, D. Demmler, T. Schneider, and O. Tkachenko, "Motion–a framework for mixed-protocol multi-party computation," *ACM Trans. Privacy Secur.*, vol. 25, no. 2, pp. 1–35, 2022.
[51] T. Schneider, A. Suresh, and H. Yalame, "Comments on 0privacy-enhanced federated learning against poisoning adversaries," *IEEE Trans. Inf. Forensics Secur.*, vol. 18, pp. 1407–1409, 2023.
[52] J. Wu, W. Zhang, and F. Luo, "On the security of "LSFL: A lightweight and secure federated learning scheme for edge computing," *IEEE Trans. Inf. Forensics Secur.*, vol. 19, pp. 3481–3482, 2024.
[53] V. Shejwalkar and A. Houmansadr, "Manipulating the byzantine: Optimizing model poisoning attacks and defenses for federated learning," in *Proc. Netw. Distrib. Syst. Secur. Symp.*, 2021. [Online]. Available: https://www.ndss-symposium.org/wp-content/uploads/ndss2021_6C-3_24498_paper.pdf
[54] C. J. B. Y. LeCun and C. Cortes, "Mnist handwritten digit database," 2010. [Online]. Available: http://yann.lecun.com/exdb/mnist/
[55] A. Krizhevsky, "Learning multiple layers of features from tiny images," 2009. [Online]. Available: http://www.cs.toronto.edu/kriz/cifar.html
[56] T.-M. H. Hsu, H. Qi, and M. Brown, "Measuring the effects of non-identical data distribution for federated visual classification," 2019, *arXiv: 1909.06335*.
[57] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proc. IEEE*, vol. 86, no. 11, pp. 2278–2324, Nov. 1998.
[58] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "ImageNet classification with deep convolutional neural networks," *Commun. ACM*, vol. 60, no. 6, pp. 84–90, 2017.
[59] C. Fung, C. J. Yoon, and I. Beschastnikh, "The limitations of federated learning in sybil settings," in *Proc. Int. Symp. Res. Attacks Intrusions Defenses*2020, pp. 301–316.

Authorized licensed use limited to: Anhui University. Downloaded on June 16,2026 at 03:35:52 UTC from IEEE Xplore. Restrictions apply.

[60] N. Wang, Y. Xiao, Y. Chen, Y. Hu, W. Lou, and Y. T. Hou, "FLARE: Defending federated learning against model poisoning attacks via latent space representations," in *Proc. ACM Asia Conf. Comput. Commun. Secur.*, 2022, pp. 946–958.
[61] T. D. Nguyen et al., "{FLAME}: Taming backdoors in federated learning," in *Proc. USENIX Secur. Symp.*, 2022, pp. 1415–1432.
[62] I. Damgård and M. Jurik, "A generalisation, a simplification and some applications of paillier's probabilistic public-key system," in *Public Key Cryptogr.*, 2001, pp. 119–136.



Jiahui Wu received the PhD degree in signal and information processing from Southwest University, Chongqing, China, in 2021. She is currently a postdoctoral fellow with the Department of New Networks, Peng Cheng Laboratory, Shenzhen, China. Her research interests include privacy-preserving machine learning, cryptography, and cloud computing security.



Fucai Luo received the PhD degree in cyber security from the State Key Laboratory of Information Security, School of Cyber Security, University of Chinese Academy of Sciences, China, in 2019. He is currently an associate researcher with the School of Computer Science and Technology, Zhejiang Gongshang University, Hangzhou, China. His research interests include lattice-based cryptography, especially fully homomorphic encryption. His recent work has focused on secure multiparty computation, proxy re-encryption, public-key encryption with keyword search, attribute-based encryption/signature.



Tiecheng Sun (Member, IEEE) received the BE and PhD degrees from the University of Electronic Science and Technology of China. In 2022, he joined Huawei as a postdoctoral researcher. In 2024, he joined the Pengcheng Laboratory as an assistant researcher. In 2025, he joined Great Wall Motors as a technical expert to be responsible for the world model algorithms. His research interests include federated learning, 3D descriptor, 3D representation, 3D reconstruction, scene understanding, world model, and associated privacy concerns.



Haiyan Wang received the PhD degree in cryptography from the State Key Laboratory of Information Security, Institute of Information Engineering, Chinese Academy of Sciences, Beijing, China, in 2017. She is currently with the Department of New Networks, Pengcheng Laboratory, Shenzhen, China. Her research interests include applied cryptography and secure multiparty computation.



Weizhe Zhang (Senior Member, IEEE) received the BEng, MEng, and PhD degrees in computer science and technology from the Harbin Institute of Technology, China, in 1999, 2001, and 2006, respectively. He is currently a professor and the vice dean with the School of Cyberspace Science Faculty of Computing, Harbin Institute of Technology, Heilongjiang, China, and the vice dean with the Department of New Networks, Peng Cheng Laboratory, Shenzhen, China. He has authored or coauthored more than 130 academic papers in journals, books, and conference proceedings. His research interests include cyberspace security, cloud computing, and high performance computing. He is a lifetime member of the ACM.

Authorized licensed use limited to: Anhui University. Downloaded on June 16,2026 at 03:35:52 UTC from IEEE Xplore. Restrictions apply.